

```
conda install -c conda-forge pymongo
conda install -c conda-forge dnspython
```

`keys.py`

The `ch16` examples folder's `TwitterMongoDB` subfolder contains this example's code and `keys.py` file. Edit this file to include your Twitter credentials and your OpenMapQuest key from the “Data Mining Twitter” chapter. After we discuss creating a MongoDB Atlas cluster, you'll also need to add your MongoDB connection string to this file.

16.4.1 Creating the MongoDB Atlas Cluster

To sign up for a free account go to

```
https://mongodb.com
```

then enter your email address and click **Get started free**. On the next page, enter your name and create a password, then read their terms of service. If you agree, click **Get started free** on this page and you'll be taken to the screen for setting up your cluster. Click **Build my first cluster** to get started.

They walk you through the getting started steps with popup bubbles that describe and point you to each task you need to complete. They provide default settings for their free Atlas cluster (**Mo** as they refer to it), so just give your cluster a name in the **Cluster Name** section, then click **Create Cluster**. At this point, they'll take you to the **Clusters** page and begin creating your new cluster, which takes several minutes.

Next, a **Connect to Atlas** popup tutorial will appear, showing a checklist of additional steps required to get you up and running:

- **Create your first database user**—This enables you to log into your cluster.
- **Whitelist your IP address**—This is a security measure which ensures that only IP addresses you verify are allowed to interact with your cluster. To connect to this cluster from multiple locations (school, home, work, etc.), you'll need to whitelist each IP address from which you intend to connect.
- **Connect to your cluster**—In this step, you'll locate your cluster's connection string, which will enable your Python code to connect to the server.

Creating Your First Database User

In the popup tutorial window, click **Create your first database user** to continue the tutorial, then follow the on-screen prompts to view the cluster's **Security** tab and click + **ADD NEW USER**. In the **Add New User** dialog, create a username and password. Write these down—you'll need them momentarily. Click **Add User** to return to the **Connect to Atlas** popup tutorial.

Whitelist Your IP Address

In the popup tutorial window, click **Whitelist your IP address** to continue the tutorial, then follow the on-screen prompts to view the cluster's **IP Whitelist** and click + **ADD IP**

ADDRESS. In the **Add Whitelist Entry** dialog, you can either add your computer's current IP address or allow access from anywhere, which they do not recommend for production databases, but is OK for learning purposes. Click **ALLOW ACCESS FROM ANYWHERE** then click **Confirm** to return to the **Connect to Atlas** popup tutorial.

Connect to Your Cluster

In the popup tutorial window, click **Connect to your cluster** to continue the tutorial, then follow the on-screen prompts to view the cluster's **Connect to *YourClusterName*** dialog. Connecting to a MongoDB Atlas database from Python requires a connection string. To get your connection string, click **Connect Your Application**, then click **Short SRV connection string**. Your connection string will appear below **Copy the SRV address**. Click **COPY** to copy the string. Paste this string into the `keys.py` file as `mongo_connection_string`'s value. Replace "<PASSWORD>" in the connection string with your password, and replace the database name "test" with "senators", which will be the database name in this example. At the bottom of the **Connect to *YourClusterName***, click **Close**. You're now ready to interact with your Atlas cluster.

16.4.2 Streaming Tweets into MongoDB

First we'll present an interactive IPython session that connects to the MongoDB database, downloads current tweets via Twitter streaming and summarizes the top-10 senators by tweet count. Next, we'll present class `TweetListener`, which handles the incoming tweets and stores their JSON in MongoDB. Finally, we'll continue the IPython session by creating an interactive Folium map that displays information from the tweets we stored.

Use Tweepy to Authenticate with Twitter

First, let's use Tweepy to authenticate with Twitter:

[lick here to view code image](#)

```
In [1]: import tweepy, keys

In [2]: auth = tweepy.OAuthHandler(
...:     keys.consumer_key, keys.consumer_secret)
...: auth.set_access_token(keys.access_token,
...:     keys.access_token_secret)
...:
```

Next, configure the Tweepy API object to wait if our app reaches any Twitter rate limits.

[lick here to view code image](#)

```
In [3]: api = tweepy.API(auth, wait_on_rate_limit=True,
...:     wait_on_rate_limit_notify=True)
...:
```

Loading the Senators' Data

We'll use the information in the file `senators.csv` (located in the `ch16 examples` folder's `TwitterMongoDB` subfolder) to track tweets to, from and about every U.S. senator. The file contains the senator's two-letter state code, name, party, Twitter handle and Twitter ID.

Twitter enables you to follow specific users via their numeric Twitter IDs, but these must be submitted as string representations of those numeric values. So, let's load `senators.csv` into pandas, convert the `TwitterID` values to strings (using `Series` method `astype`) and display several rows of data. In this case, we set 6 as the maximum number of columns to display. Later we'll add another column to the `DataFrame` and this setting will ensure that all the columns are displayed, rather than a few with in between:

[lick here to view code image](#)

```
In [4]: import pandas as pd

In [5]: senators_df = pd.read_csv('senators.csv')

In [6]: senators_df['TwitterID'] = senators_df['TwitterID'].astype(str)

In [7]: pd.options.display.max_columns = 6

In [8]: senators_df.head()
Out[8]:
```

	State	Name	Party	TwitterHandle	TwitterID
0	AL	Richard Shelby	R	SenShelby	21111098
1	AL	Doug Jones	D	SenDougJones	941080085121175552
2	AK	Lisa Murkowski	R	lisamurkowski	18061669
3	AK	Dan Sullivan	R	SenDanSullivan	2891210047
4	AZ	Jon Kyl	R	SenJonKyl	24905240

Configuring the MongoClient

To store the tweet's JSON as documents in a MongoDB database, you must first connect to your MongoDB Atlas cluster via a pymongo `MongoClient`, which receives your cluster's connection string as its argument:

[lick here to view code image](#)

```
In [9]: from pymongo import MongoClient

In [10]: atlas_client = MongoClient(keys.mongo_connection_string)
```

Now, we can get a pymongo `Database` object representing the `senators` database. The following statement creates the database if it does not exist:

[lick here to view code image](#)

```
In [11]: db = atlas_client.senators
```

Setting up Tweet Stream

Let's specify the number of tweets to download and create the `TweetListener`. We pass the `db` object representing the MongoDB database to the `TweetListener` so it can write the tweets into the database. Depending on the rate at which people are tweeting about the senators, it may take minutes to hours to get 10,000 tweets. For testing purposes, you might want to use a smaller number:

[lick here to view code image](#)

```

In [12]: from tweetlistener import TweetListener

In [13]: tweet_limit = 10000

In [14]: twitter_stream = tweepy.Stream(api.auth,
...:     TweetListener(api, db, tweet_limit))
...:

```

Starting the Tweet Stream

Twitter live streaming allows you to track up to 400 keywords and follow up to 5,000 Twitter IDs at a time. In this case, let's track the senators' Twitter handles and follow the senator's Twitter IDs. This should give us tweets from, to and about each senator. To show you progress, we display the screen name and time stamp for each tweet received, and the total number of tweets so far. To save space, we show here only one of those tweet outputs and replace the user's screen name with XXXXXXXX:

[lick here to view code image](#)

```

In [15]: twitter_stream.filter(track=senators_df.TwitterHandle.tolist(),
...:     follow=senators_df.TwitterID.tolist())
...:
Screen name: XXXXXXXX
Created at: Sun Dec 16 17:19:19 +0000 2018
Tweets received: 1
...

```

Class TweetListener

For this example, we slightly modified class TweetListener from the “Data Mining Twitter” chapter. Much of the Twitter and Tweepy code shown below is identical to the code you saw previously, so we'll focus on only the new concepts here:

[lick here to view code image](#)

```

1 # tweetlistener.py
2 """TweetListener downloads tweets and stores them in MongoDB."""
3 import json
4 import tweepy
5
6 class TweetListener(tweepy.StreamListener):
7     """Handles incoming Tweet stream."""
8
9     def __init__(self, api, database, limit=10000):
10         """Create instance variables for tracking number of tweets."""
11         self.db = database
12         self.tweet_count = 0
13         self.TWEET_LIMIT = limit # 10,000 by default
14         super().__init__(api) # call superclass's init
15
16     def on_connect(self):
17         """Called when your connection attempt is successful, enabling
18         you to perform appropriate application tasks at that point."""
19         print('Successfully connected to Twitter\n')
20
21     def on_data(self, data):
22         """Called when Twitter pushes a new tweet to you."""
23         self.tweet_count += 1 # track number of tweets processed
24         json_data = json.loads(data) # convert string to JSON

```

```

25         self.db.tweets.insert_one(json_data)      # store in tweets collection
26         print(f'      Screen name: {json_data["user"]["name"]}')
27         print(f'      Created at: {json_data["created_at"]}')
28         print(f'Tweets received: {self.tweet_count}')
29
30         # if TWEET_LIMIT is reached, return False to terminate streaming
31         return self.tweet_count != self.TWEET_LIMIT
32
33     def on_error(self, status):
34         print(status)
35         return True

```

Previously, `TweetListener` overrode method `on_status` to receive Tweepy `Status` objects representing tweets. Here, we override the `on_data` method instead (lines 21–31). Rather than `Status` objects, `on_data` receives each tweet object's *raw JSON*. Line 24 converts the JSON string received by `on_data` into a Python JSON object. Each MongoDB database contains one or more **Collections** of documents. In line 25, the expression

```
self.db.tweets
```

accesses the Database object `db`'s `tweets` Collection, creating it if it does not already exist. Line 25 uses the `tweets` Collection's **`insert_one` method** to store the JSON object in the `tweets` collection.

Counting Tweets for Each Senator

Next, we'll perform a full-text search on the collection of tweets and count the number of tweets containing each senator's Twitter handle. To text search in MongoDB, you must create a *text index* for the collection.⁶ This specifies which document field(s) to search. Each text index is defined as a tuple containing the field name to search and the index type (`'text'`). MongoDB's wildcard specifier (`$**`) indicates that every text field in a document (a JSON tweet object in our case) should be indexed for a full-text search:

⁶For additional details on MongoDB index types, text indexes and operators, see:

<https://docs.mongodb.com/manual/indexes>,

<https://docs.mongodb.com/manual/core/index-text> and

<https://docs.mongodb.com/manual/reference/operator>.

[click here to view code image](#)

```

In [16]: db.tweets.create_index([('**', 'text')])
Out[16]: '**_text'

```

Once the index is defined, we can use the Collection's **`count_documents` method** to count the total number of documents in the collection that contain the specified text. Let's search the database's `tweets` collection for every twitter handle in the `senators_df` DataFrame's `TwitterHandle` column:

[click here to view code image](#)

```
In [17]: tweet_counts = []
```

```
In [18]: for senator in senators_df.TwitterHandle:
...:     tweet_counts.append(db.tweets.count_documents(
...:         {"$text": {"$search": senator}}))
...:
```

The JSON object passed to `count_documents` in this case indicates that we’re using the index named `text` to search for the value of `senator`.

Show Tweet Counts for Each Senator

Let’s create a copy of the DataFrame `senators_df` that contains the `tweet_counts` as a new column, then display the top-10 senators by tweet count:

[lick here to view code image](#)

```
In [19]: tweet_counts_df = senators_df.assign(Tweets=tweet_counts)

In [20]: tweet_counts_df.sort_values(by='Tweets',
...:     ascending=False).head(10)
...:
```

Out[20]:

	State	Name	Party	TwitterHandle	TwitterID	Tweets
78	SC	Lindsey Graham	R	LindseyGrahamSC	432895323	1405
41	MA	Elizabeth Warren	D	SenWarren	970207298	1249
8	CA	Dianne Feinstein	D	SenFeinstein	476256944	1079
20	HI	Brian Schatz	D	brianschatz	47747074	934
62	NY	Chuck Schumer	D	SenSchumer	17494010	811
24	IL	Tammy Duckworth	D	SenDuckworth	1058520120	656
13	CT	Richard Blumenthal	D	SenBlumenthal	278124059	646
21	HI	Mazie Hirono	D	maziehirono	92186819	628
86	UT	Orrin Hatch	R	SenOrrinHatch	262756641	506
77	RI	Sheldon Whitehouse	D	SenWhitehouse	242555999	350

Get the State Locations for Plotting Markers

Next, we’ll use the techniques you learned in the “Data Mining Twitter” chapter to get each state’s latitude and longitude coordinates. We’ll soon use these to place on a Folium map popup markers that contain the names and numbers of tweets mentioning each state’s senators.

The file `state_codes.py` contains a `state_codes` dictionary that maps two-letter state codes to their full state names. We’ll use the full state names with `geopy`’s `OpenMapQuest` `geocode` function to look up the location of each state.⁷ First, let’s import the libraries we need and the `state_codes` dictionary:

⁷We use full state names because, during our testing, the two-letter state codes did not always return correct locations.

[lick here to view code image](#)

```
In [21]: from geopy import OpenMapQuest

In [22]: import time

In [23]: from state_codes import state_codes
```

Next, let's get the `geocoder` object to translate location names into `Location` objects:

[lick here to view code image](#)

```
In [24]: geo = OpenMapQuest(api_key=keys.mapquest_key)
```

There are two senators from each state, so we can look up each state's location once and use the `Location` object for both senators from that state. Let's get the unique state names, then sort them into ascending order:

[lick here to view code image](#)

```
In [25]: states = tweet_counts_df.State.unique()
```

```
In [26]: states.sort()
```

The next two snippets use code from the “Data Mining Twitter” chapter to look up each state's location. In snippet [28], we call the `geocode` function with the state name followed by `', USA'` to ensure that we get United States locations,⁸ since there are places outside the United States with the same names as U.S. states. To show progress, we display each new `Location` object's string:

⁸When we initially performed the geocoding for Washington state, OpenMapQuest returned Washington, D.C.s location. So we modified `state_codes.py` to use Washington State instead.

[lick here to view code image](#)

```
In [27]: locations = []

In [28]: for state in states:
...:     processed = False
...:     delay = .1
...:     while not processed:
...:         try:
...:             locations.append(
...:                 geo.geocode(state_codes[state] + ', USA'))
...:             print(locations[-1])
...:             processed = True
...:         except: # timed out, so wait before trying again
...:             print('OpenMapQuest service timed out. Waiting.')
...:             time.sleep(delay)
...:             delay += .1
...:
Alaska, United States of America
Alabama, United States of America
Arkansas, United States of America
...
```

Grouping the Tweet Counts by State

We'll use the total number of tweets for the two senators in a state to color that state on the map. Darker colors will represent the states with higher tweet counts. To prepare the data for mapping, let's use the pandas `DataFrame` method **`groupby`** to group the senators by state and calculate the total tweets by state:

[lick here to view code image](#)

```
In [29]: tweets_counts_by_state = tweet_counts_df.groupby(
...:     'State', as_index=False).sum()
...:

In [30]: tweets_counts_by_state.head()
Out[30]:
```

	State	Tweets
0	AK	27
1	AL	2
2	AR	47
3	AZ	47
4	CA	1135

The `as_index=False` keyword argument in snippet [29] indicates that the state codes should be values in a column of the resulting **GroupBy** object, rather than the indices for the rows. The **GroupBy** object's `sum` method totals the numeric data (the tweets by state). Snippet [30] displays several rows of the **GroupBy** object so you can see some of the results.

Creating the Map

Next, let's create the map. You may want to adjust the zoom. On our system, the following snippet creates a map in which we initially can see only the continental United States. Remember that Folium maps are interactive, so once the map is displayed, you can scroll to zoom in and out or drag to see different areas, such as Alaska or Hawaii:

[lick here to view code image](#)

```
In [31]: import folium

In [32]: usmap = folium.Map(location=[39.8283, -98.5795],
...:     zoom_start=4, detect_retina=True,
...:     tiles='Stamen Toner')
...:
```

Creating a Choropleth to Color the Map

A **choropleth** shades areas in a map using the values you specify to determine color. Let's create a choropleth that colors the states by the number of tweets containing their senators' Twitter handles. First, save Folium's `us-states.json` file at

```
https://raw.githubusercontent.com/python-visualization/folium/master/examples/d
```

o the folder containing this example. This file contains a JSON dialect called **GeoJSON (Geographic JSON)** that describes the boundaries of shapes—in this case, the boundaries of every U.S. state. The choropleth uses this information to shade each state. For more about GeoJSON, see <http://geojson.org/>.⁹ The following snippets create the choropleth, then add it to the map:

⁹Folium provides several other GeoJSON files in its examples folder at <https://github.com/python-visualization/folium/tree/master/examples/data>. You also can create your own

at <http://geojson.io>.

[lick here to view code image](#)

```
In [33]: choropleth = folium.Choropleth(
...:     geo_data='us-states.json',
...:     name='choropleth',
...:     data=tweets_counts_by_state,
...:     columns=['State', 'Tweets'],
...:     key_on='feature.id',
...:     fill_color='YlOrRd',
...:     fill_opacity=0.7,
...:     line_opacity=0.2,
...:     legend_name='Tweets by State'
...: ).add_to(usmap)
...:

In [34]: layer = folium.LayerControl().add_to(usmap)
```

In this case, we used the following arguments:

- `geo_data='us-states.json'`—This is the file containing the GeoJSON that specifies the shapes to color.
- `name='choropleth'`—Folium displays the `Choropleth` as a layer over the map. This is the name for that layer that will appear in the map's layer controls, which enable you to hide and show the layers. These controls appear when you click the layers icon () on the map.
- `data=tweets_counts_by_state`—This is a pandas `DataFrame` (or `Series`) containing the values that determine the `Choropleth` colors.
- `columns=['State', 'Tweets']`—When the data is a `DataFrame`, this is a list of two columns representing the keys and the corresponding values used to color the `Choropleth`.
- `key_on='feature.id'`—This is a variable in the GeoJSON file to which the `Choropleth` binds the values in the `columns` argument.
- `fill_color='YlOrRd'`—This is a color map specifying the colors to use to fill in the states. Folium provides 12 colormaps: 'BuGn', 'BuPu', 'GnBu', 'OrRd', 'PuBu', 'PuBuGn', 'PuRd', 'RdPu', 'YlGn', 'YlGnBu', 'YlOrBr' and 'YlOrRd'. You should experiment with these to find the most effective and eye-pleasing ones for your application(s).
- `fill_opacity=0.7`—A value from 0.0 (transparent) to 1.0 (opaque) specifying the transparency of the fill colors displayed in the states.
- `line_opacity=0.2`—A value from 0.0 (transparent) to 1.0 (opaque) specifying the transparency of lines used to delineate the states.
- `legend_name='Tweets by State'`—At the top of the map, the `Choropleth` displays a color bar (the legend) indicating the value range represented by the colors. This `legend_name` text appears below the color bar to indicate what the colors represent.

The complete list of Choropleth keyword arguments is documented at:

```
http://python-visualization.github.io/folium/modules.html#folium.features.Choro
```

reating the Map Markers for Each State

Next, we'll create `Markers` for each state. To ensure that the senators are displayed in descending order by the number of tweets in each state's `Marker`, let's sort `tweet_counts_df` in descending order by the 'Tweets' column:

[lick here to view code image](#)

```
In [35]: sorted_df = tweet_counts_df.sort_values(
...:     by='Tweets', ascending=False)
...:
```

The loop in the following snippet creates the `Markers`. First,

```
sorted_df.groupby('State')
```

groups `sorted_df` by 'State'. A `DataFrame`'s `groupby` method maintains the *original row order* in each group. Within a given group, the senator with the most tweets will be first, because we sorted the senators in descending order by tweet count in snippet [35]:

[lick here to view code image](#)

```
In [36]: for index, (name, group) in enumerate(sorted_df.groupby('State')):
...:     strings = [state_codes[name]] # used to assemble popup text
...:
...:     for s in group.itertuples():
...:         strings.append(
...:             strings.append(
...:
...:         text = '<br>'.join(strings)
...:         marker = folium.Marker(
...:             (locations[index].latitude, locations[index].longitude),
...:             popup=text)
...:         marker.add_to(usmap)
...:
...:
```

We pass the grouped `DataFrame` to `enumerate`, so we can get an index for each group, which we'll use to look up each state's `Location` in the `locations` list. Each group has a name (the state code we grouped by) and a collection of items in that group (the two senators for that state). The loop operates as follows:

- We look up the full state name in the `state_codes` dictionary, then store it in the `strings` list—we'll use this list to assemble the `Marker`'s popup text.
- The nested loop walks through the items in the `group` collection, returning each as a named tuple that contains a given senator's data. We create a formatted string for the current senator containing the person's name, party and number of tweets, then append

that to the `strings` list.

- The `Marker` text can use HTML for formatting. We join the `strings` list's elements, separating each from the next with an HTML `<br>` element which creates a new line in HTML.
- We create the `Marker`. The first argument is the `Marker`'s location as a tuple containing the latitude and longitude. The `popup` keyword argument specifies the text to display if the user clicks the `Marker`.
- We add the `Marker` to the map.

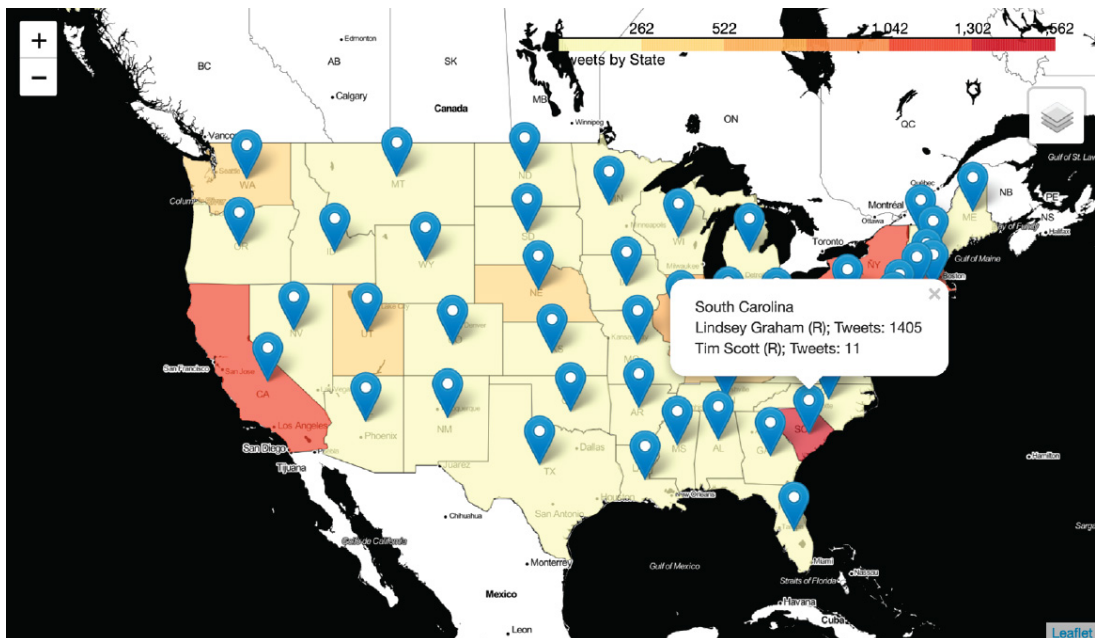
Displaying the Map

Finally, let's save the map into an HTML file

[lick here to view code image](#)

```
In [17]: usmap.save('SenatorsTweets.html')
```

Open the HTML file in your web browser to view and interact with the map. Recall that you can drag the map to see Alaska and Hawaii. Here we show the popup text for the South Carolina marker:



You could enhance this case study to use the sentiment-analysis techniques you learned in previous chapters to rate as positive, neutral or negative the sentiment expressed by people who send tweets ("tweeters") mentioning each senator's handle.

16.5 HADOOP

The next several sections show how Apache Hadoop and Apache Spark deal with big-data storage and processing challenges via huge clusters of computers, massively parallel processing, Hadoop MapReduce programming and Spark in-memory processing techniques. Here, we discuss Apache Hadoop, a key big-data infrastructure technology that also serves as the foundation for many recent advancements in big-data processing and an entire ecosystem

f software tools that are continually evolving to support today's big-data needs.

16.5.1 Hadoop Overview

When Google was launched in 1998, the amount of online data was already enormous with approximately 2.4 million websites ⁰—truly big data. Today there are now nearly two billion websites ¹ (almost a thousandfold increase) and Google is handling over two trillion searches per year! ² Having used Google search since its inception, our sense is that today's responses are significantly faster.

⁰ <http://www.internetlivestats.com/total-number-of-websites/>.

¹ <http://www.internetlivestats.com/total-number-of-websites/>.

² <http://www.internetlivestats.com/google-search-statistics/>.

When Google was developing their search engine, they knew that they needed to return search results quickly. The only practical way to do this was to store and index the entire Internet using a clever combination of secondary storage and main memory. Computers of that time couldn't hold that amount of data and could not analyze that amount of data fast enough to guarantee prompt search-query responses. So Google developed a **clustering** system, tying together vast numbers of computers—called **nodes**. Because having more computers and more connections between them meant greater chance of hardware failures, they also built in high levels of *redundancy* to ensure that the system would continue functioning even if nodes within clusters failed. The data was distributed across all these inexpensive “commodity computers.” To satisfy a search request, all the computers in the cluster searched in parallel the portion of the web they had locally. Then the results of those searches were gathered up and reported back to the user.

To accomplish this, Google needed to develop the clustering hardware and software, including distributed storage. Google publishes its designs, but did not open source its software. Programmers at Yahoo!, working from Google's designs in the “Google File System” paper, ³ then built their own system. They open-sourced their work and the Apache organization implemented the system as Hadoop. The name came from an elephant stuffed animal that belonged to a child of one of Hadoop's creators.

³ <http://static.googleusercontent.com/media/research.google.com/en//archive/gfosp2003.pdf>.

Two additional Google papers also contributed to the evolution of Hadoop—“MapReduce: Simplified Data Processing on Large Clusters” ⁴ and “Bigtable: A Distributed Storage System for Structured Data,” ⁵ which was the basis for Apache HBase (a NoSQL key-value and column--based database). ⁶

⁴ <http://static.googleusercontent.com/media/research.google.com/en//archive/masdi04.pdf>.

⁵ <http://static.googleusercontent.com/media/research.google.com/en//archive/bisdi06.pdf>.

⁶Many other influential big-data-related papers (including the ones we mentioned) can be

found at: <https://bigdata-madesimple.com/research-papers-that-changed-the-world-of-big-data/>.

DFS, MapReduce and YARN

Hadoop's key components are:

- **HDFS (Hadoop Distributed File System)** for storing massive amounts of data throughout a cluster, and
- **MapReduce** for implementing the tasks that process the data.

Earlier in the book we introduced basic functional-style programming and filter/map/reduce. Hadoop MapReduce is similar in concept, just on a massively parallel scale. A MapReduce task performs two steps—**mapping** and **reduction**. The mapping step, which also may include *filtering*, processes the original data across the entire cluster and maps it into tuples of key–value pairs. The reduction step then combines those tuples to produce the results of the MapReduce task. The key is how the MapReduce step is performed. Hadoop divides the data into *batches* that it distributes across the nodes in the cluster—anywhere from a few nodes to a Yahoo! cluster with 40,000 nodes and over 100,000 cores.⁷ Hadoop also distributes the MapReduce task's code to the nodes in the cluster and executes the code in parallel on every node. Each node processes only the batch of data stored on that node. The reduction step combines the results from all the nodes to produce the final result. To coordinate this, Hadoop uses **YARN** (“yet another resource negotiator”) to manage all the resources in the cluster and schedule tasks for execution.

⁷ <https://wiki.apache.org/hadoop/PoweredBy>.

Hadoop Ecosystem

Though Hadoop began with HDFS and MapReduce, followed closely by YARN, it has grown into a large ecosystem that includes Spark (discussed in sections 16.6– 16.7) and many other Apache projects:^{8, 9, 10}

⁸ <https://hortonworks.com/ecosystems/>.

⁹ <https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/>.

¹⁰ <https://www.janbasktraining.com/blog/introduction-architecture-components-hadoop-ecosystem/>.

- **Ambari** (<https://ambari.apache.org>)—Tools for managing Hadoop clusters.
- **Drill** (<https://drill.apache.org>)—SQL querying of non-relational data in Hadoop and NoSQL databases.
- **Flume** (<https://flume.apache.org>)—A service for collecting and storing (in HDFS and other storage) streaming event data, like high-volume server logs, IoT messages and more.
- **HBase** (<https://hbase.apache.org>)—A NoSQL database for big data with “billions

rows by ¹ millions of columns—atop clusters of commodity hardware.”

¹We used the word by to replace X in the original text.

- **Hive** (<https://hive.apache.org>)—Uses SQL to interact with data in data warehouses. A **data warehouse** aggregates data of various types from various sources. Common operations include extracting data, transforming it and loading (known as **ETL**) into another database, typically so you can analyze it and create reports from it.
- **Impala** (<https://impala.apache.org>)—A database for real-time SQL-based queries across distributed data stored in Hadoop HDFS or HBase.
- **Kafka** (<https://kafka.apache.org>)—Real-time messaging, stream processing and storage, typically to transform and process high-volume streaming data, such as website activity and streaming IoT data.
- **Pig** (<https://pig.apache.org>)—A scripting platform that converts data analysis tasks from a scripting language called **Pig Latin** into MapReduce tasks.
- **Sqoop** (<https://sqoop.apache.org>)—Tool for moving structured, semi-structured and unstructured data between databases.
- **Storm** (<https://storm.apache.org>)—A real-time stream-processing system for tasks such as data analytics, machine learning, ETL and more.
- **ZooKeeper** (<https://zookeeper.apache.org>)—A service for managing cluster configurations and coordination between clusters.
- And more.

Hadoop Providers

Numerous cloud vendors provide Hadoop as a service, including Amazon EMR, Google Cloud DataProc, IBM Watson Analytics Engine, Microsoft Azure HDInsight and others. In addition, companies like Cloudera and Hortonworks (which at the time of this writing are merging) offer integrated Hadoop-ecosystem components and tools via the major cloud vendors. They also offer free *downloadable* environments that you can run on the desktop ² for learning, development and testing before you commit to cloud-based hosting, which can incur significant costs. We introduce MapReduce programming in the example in the following sections by using a Microsoft cloud-based Azure HDInsight cluster, which provides Hadoop as a service.

²Check their significant system requirements first to ensure that you have the disk space and memory required to run them.

Hadoop 3

Apache continues to evolve Hadoop. Hadoop 3 ³ was released in December of 2017 with many improvements, including better performance and significantly improved storage efficiency. ⁴

³For a list of features in Hadoop 3, see <https://hadoop.apache.org/docs/r3.0.0/>.

⁴ <https://www.datanami.com/2018/10/18/is-hadoop-officially-dead/>.

16.5.2 Summarizing Word Lengths in *Romeo and Juliet* via MapReduce

In the next several subsections, you'll create a cloud-based, multi-node cluster of computers using Microsoft Azure HDInsight. Then, you'll use the service's capabilities to demonstrate Hadoop MapReduce running on that cluster. The MapReduce task you'll define will determine the length of each word in `RomeoAndJuliet.txt` (from the “Natural Language Processing” chapter), then summarize how many words of each length there are. After defining the task's mapping and reduction steps, you'll submit the task to your HDInsight cluster, and Hadoop will decide how to use the cluster of computers to perform the task.

16.5.3 Creating an Apache Hadoop Cluster in Microsoft Azure HDInsight

Most major cloud vendors have support for Hadoop and Spark computing clusters that you can configure to meet your application's requirements. Multi-node cloud-based clusters typically are *paid* services, though most vendors provide free trials or credits so you can try out their services.

We want you to experience the process of setting up clusters and using them to perform tasks. So, in this Hadoop example, you'll use Microsoft Azure's HDInsight service to create cloud-based clusters of computers in which to test our examples. Go to

```
https://azure.microsoft.com/en-us/free
```

to sign up for an account. Microsoft requires a credit card for identity verification.

Various services are always free and some you can continue to use for 12 months. For information on these services see:

```
https://azure.microsoft.com/en-us/free/free-account-faq/
```

Microsoft also gives you a credit to experiment with their *paid* services, such as their HDInsight Hadoop and Spark services. Once your credits run out or 30 days pass (whichever comes first), you cannot continue using paid services unless you authorize Microsoft to charge your card.

Because you'll use your new Azure account's credit for these examples, ⁵ we'll discuss how to configure a low-cost cluster that uses less computing resources than Microsoft allocates by default. ⁶ **Caution: Once you allocate a cluster, it incurs costs whether you're using it or not. So, when you complete this case study, be sure to delete your cluster(s) and other resources, so you don't incur additional charges.** For more information, see:

⁵For Microsoft's latest free account features, visit <https://azure.microsoft.com/en-us/free/>.

⁶For Microsoft's recommended cluster configurations, see

<https://docs.microsoft.com/en-us/azure/hdinsight/hdinsight-component-versioning#default-node-configuration-andvirtual-machine->

izes-for-clusters. If you configure a cluster that's too small for a given scenario, when you try to deploy the cluster you'll receive an error.

```
https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po
```

or Azure-related documentation and videos, visit:

- <https://docs.microsoft.com/en-us/azure/>—the Azure documentation.
- <https://channel9.msdn.com/>—Microsoft's Channel 9 video network.
- <https://www.youtube.com/user/windowsazure>—Microsoft's Azure channel on YouTube.

Creating an HDInsight Hadoop Cluster

The following link explains how to set up a cluster for Hadoop using the Azure HDInsight service:

```
https://docs.microsoft.com/en-us/azure/hdinsight/hadoop/apache-hadoop-linux-cre
```

While following their **Create a Hadoop cluster** steps, please note the following:

- In *Step 1*, you access the Azure portal by logging into your account at

```
https://portal.azure.com
```

- In *Step 2*, **Data + Analytics** is now called **Analytics**, and the HDInsight icon and icon color have changed from what is shown in the tutorial.
- In *Step 3*, you must choose a cluster name that does *not* already exist. When you enter your cluster name, Microsoft will check whether that name is available and display a message if it is not. You must create a password. For the **Resource group**, you'll also need to click **Create new** and provide a group name. Leave all other settings in this step as is.
- In *Step 5*: Under **Select a Storage account**, click **Create new** and provide a storage account name containing only lowercase letters and numbers. Like the cluster name, the storage account name must be unique.

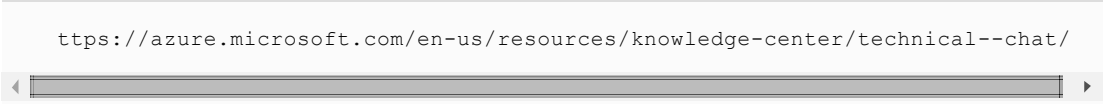
When you get to the **Cluster summary** you'll see that Microsoft initially configures the cluster as **Head (2 x D12 v2), Worker (4 x D4 v2)**. At the time of this writing, the estimated cost-per-hour for this configuration was \$3.11. This setup uses a total of 6 CPU nodes with 40 cores—far more than we need for demonstration purposes.

You can edit this setup to use fewer CPUs and cores, which also saves money. Let's change the configuration to a four-CPU cluster with 16 cores that uses less powerful computers. In the **Cluster summary**:

1. Click **Edit** to the right of **Cluster size**.
2. Change the **Number of Worker** nodes to 2.
3. Click **Worker node size**, then **View all**, select **D3 v2** (this is the minimum CPU size for Hadoop nodes) and click **Select**.
4. Click **Head node size**, then **View all**, select **D3 v2** and click **Select**.
5. Click **Next** and click **Next** again to return to the **Cluster summary**. Microsoft will validate the new configuration.
6. When the **Create** button is enabled, click it to deploy the cluster.

It takes 20–30 minutes for Microsoft to “spin up” your cluster. During this time, Microsoft is allocating all the resources and software the cluster requires.

After the changes above, our estimated cost for the cluster was \$1.18 per hour, based on *average* use for similarly configured clusters. Our actual charges were less than that. If you encounter any problems configuring your cluster, Microsoft provides HDInsight chat-based support at:



```
https://azure.microsoft.com/en-us/resources/knowledge-center/technical--chat/
```

16.5.4 Hadoop Streaming

For languages like Python that are not natively supported in Hadoop, you must use **Hadoop streaming** to implement your tasks. In Hadoop streaming, the Python scripts that implement the mapping and reduction steps use the **standard input stream** and **standard output stream** to communicate with Hadoop. Usually, the standard input stream reads from the keyboard and the standard output stream writes to the command line. However, these can be *redirected* (as Hadoop does) to read from other sources and write to other destinations. Hadoop uses the streams as follows:

- Hadoop supplies the input to the mapping script—called the **mapper**. This script reads its input from the standard input stream.
- The mapper writes its results to the standard output stream.
- Hadoop supplies the mapper’s output as the input to the reduction script—called the **reducer**—which reads from the standard input stream.
- The reducer writes its results to the standard output stream.
- Hadoop writes the reducer’s output to the Hadoop file system (HDFS).

The mapper and reducer terminology used above should sound familiar to you from our discussions of functional-style programming and filter, map and reduce in the “Sequences: Lists and Tuples” chapter.

16.5.5 Implementing the Mapper

In this section, you'll create a mapper script that takes lines of text as input from Hadoop and maps them to *key-value pairs* in which each key is a word, and its corresponding value is 1. The mapper sees each word individually so, as far as it is concerned, there's only one of each word. In the next section, the reducer will summarize these key-value pairs by key, reducing the counts to a single count for each key. By default, Hadoop expects the mapper's output and the reducer's input and output to be in the form of key-value pairs separated by a *tab*.

In the mapper script (`length_mapper.py`), the notation `#!` in line 1 tells Hadoop to execute the Python code using `python3`, rather than the default Python 2 installation. This line must come before all other comments and code in the file. At the time of this writing, Python 2.7.12 and Python 3.5.2 were installed. Note that because the cluster does not have Python 3.6 or higher, you cannot use f-strings in your code.

[lick here to view code image](#)

```
1 #!/usr/bin/env python3
2 # length_mapper.py
3 """Maps lines of text to key-value pairs of word lengths and 1."""
4 import sys
5
6 def tokenize_input():
7     """Split each line of standard input into a list of strings."""
8     for line in sys.stdin:
9         yield line.split()
10
11 # read each line in the the standard input and for every word
12 # produce a key-value pair containing the word, a tab and 1
13 for line in tokenize_input():
14     for word in line:
15         print(str(len(word)) + '\t1')
```

Generator function `tokenize_input` (lines 6–9) reads lines of text from the standard input stream and for each returns a list of strings. For this example, we are not removing punctuation or stop words as we did in the “Natural Language Processing” chapter.

When Hadoop executes the script, lines 13–15 iterate through the lists of strings from `tokenize_input`. For each list (`line`) and for every string (`word`) in that list, line 15 outputs a key-value pair with the word's length as the key, a tab (`\t`) and the value 1, indicating that there is one word (so far) of that length. Of course, there probably are many words of that length. The MapReduce algorithm's reduction step will summarize these key-value pairs, reducing all those with the same key to a single key-value pair with the total count.

16.5.6 Implementing the Reducer

In the reducer script (`length_reducer.py`), function `tokenize_input` (lines 8–11) is a generator function that reads and splits the key-value pairs produced by the mapper. Again, the MapReduce algorithm supplies the standard input. For each line, `tokenize_input` strips any leading or trailing whitespace (such as the terminating newline) and yields a list containing the key and a value.

[lick here to view code image](#)

```

1 #!/usr/bin/env python3
2 # length_reducer.py
3 """Counts the number of words with each length."""
4 import sys
5 from itertools import groupby
6 from operator import itemgetter
7
8 def tokenize_input():
9     """Split each line of standard input into a key and a value."""
10    for line in sys.stdin:
11        yield line.strip().split('\t')
12
13 # produce key-value pairs of word lengths and counts separated by tabs
14 for word_length, group in groupby(tokenize_input(), itemgetter(0)):
15     try:
16         total = sum(int(count) for word_length, count in group)
17         print(word_length + '\t' + str(total))
18     except ValueError:
19         pass # ignore word if its count was not an integer

```

When the MapReduce algorithm executes this reducer, lines 14–19 use the **groupby** function from the **itertools module** to group all word lengths of the same value:

- The first argument calls `tokenize_input` to get the lists representing the key–value pairs.
- The second argument indicates that the key–value pairs should be grouped based on the element at index 0 in each list—that is the key.

Line 16 totals all the counts for a given key. Line 17 outputs a new key–value pair consisting of the word and its total. The MapReduce algorithm takes all the final word-count outputs and writes them to a file in HDFS—the Hadoop file system.

16.5.7 Preparing to Run the MapReduce Example

Next, you’ll upload files to the cluster so you can execute the example. In a Command Prompt, Terminal or shell, change to the folder containing your mapper and reducer scripts and the `RomeoAndJuliet.txt` file. We assume all three are in this chapter’s `ch16` examples folder, so be sure to copy your `RomeoAndJuliet.txt` file to this folder first.

Copying the Script Files to the HDInsight Hadoop Cluster

Enter the following command to upload the files. Be sure to replace *YourClusterName* with the cluster name you specified when setting up the Hadoop cluster and press *Enter* only after you’ve typed the entire command. The colon in the following command is required and indicates that you’ll supply your cluster password when prompted. At that prompt, type the password you specified when setting up the cluster, then press *Enter*:

[lick here to view code image](#)

```

scp length_mapper.py length_reducer.py RomeoAndJuliet.txt
sshuser@YourClusterName-ssh.azurehdinsight.net:

```

The first time you do this, you’ll be asked for security reasons to confirm that you trust the

target host (that is, Microsoft Azure).

Copying RomeoAndJuliet into the Hadoop File System

For Hadoop to read the contents of `RomeoAndJuliet.txt` and supply the lines of text to your mapper, you must first copy the file into Hadoop's file system. First, you must use `ssh` ⁷ to log into your cluster and access its command line. In a Command Prompt, Terminal or shell, execute the following command. Be sure to replace *YourClusterName* with your cluster name. Again, you'll be prompted for your cluster password:

⁷Windows users: If `ssh` does not work for you, install and enable it as described at <https://blogs.msdn.microsoft.com/powershell/2017/12/15/using-the-openssh-beta-in-windows-10-fall-creators-update-and-windows-server-1709/>. After completing the installation, log out and log back in or restart your system to enable `ssh`.

[lick here to view code image](#)

```
ssh sshuser@YourClusterName-ssh.azurehdinsight.net
```

For this example, we'll use the following Hadoop command to copy the text file into the already existing folder `/examples/data` that the cluster provides for use with Microsoft's Azure Hadoop tutorials. Again, press *Enter* only when you've typed the entire command:

[lick here to view code image](#)

```
hadoop fs -copyFromLocal RomeoAndJuliet.txt
/example/data/RomeoAndJuliet.txt
```

16.5.8 Running the MapReduce Job

Now you can run the MapReduce job for `RomeoAndJuliet.txt` on your cluster by executing the following command. For your convenience, we provided the text of this command in the file `yarn.txt` with this example, so you can copy and paste it. We reformatted the command here for readability:

[lick here to view code image](#)

```
yarn jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar
-D mapred.output.key.comparator.class=
  org.apache.hadoop.mapred.lib.KeyFieldBasedComparator
-D mapred.text.key.comparator.options=-n
-files length_mapper.py,length_reducer.py
-mapper length_mapper.py
-reducer length_reducer.py
-input /example/data/RomeoAndJuliet.txt
-output /example/wordlengthsoutput
```

The **yarn command** invokes the Hadoop's YARN ("yet another resource negotiator") tool to manage and coordinate access to the Hadoop resources the MapReduce task uses. The file `hadoop-streaming.jar` contains the Hadoop streaming utility that allows you to use Python to implement the mapper and reducer. The two `-D` options set Hadoop properties

that enable it to sort the final key–value pairs by key (`KeyFieldBasedComparator`) in descending order numerically (`-n`; the minus indicates descending order) rather than alphabetically. The other command-line arguments are:

- `-files`—A comma-separated list of file names. Hadoop copies these files to every node in the cluster so they can be executed locally on each node.
- `-mapper`—The name of the mapper’s script file.
- `-reducer`—The name of the reducer’s script file
- `-input`—The file or directory of files to supply as input to the mapper.
- `-output`—The HDFS directory in which the output will be written. If this folder already exists, an error will occur.

The following output shows some of the feedback that Hadoop produces as the MapReduce job executes. We replaced chunks of the output with `to save space` and bolded several lines of interest including:

- The total number of “input paths to process”—the 1 source of input in this example is the `RomeoAndJuliet.txt` file.
- The “number of splits”—2 in this example, based on the number of worker nodes in our cluster.
- The percentage completion information.
- File System Counters, which include the numbers of bytes read and written.
- Job Counters, which show the number of mapping and reduction tasks used and various timing information.
- Map-Reduce Framework, which shows various information about the steps performed.

[lick here to view code image](#)

```
ackageJobJar: []      [/usr/hdp/2.6.5.3004-13/hadoop-mapreduce/hadoop-streaming-2.
...
18/12/05 16:46:25 INFO  mapred.FileInputFormat: Total input paths to process :
18/12/05 16:46:26 INFO  mapreduce.JobSubmitter: number of splits:2
...
18/12/05 16:46:26 INFO mapreduce.Job: The url to track the job: http://hn0-p
...
18/12/05 16:46:35 INFO mapreduce.Job: map    0% reduce 0%
18/12/05 16:46:43 INFO mapreduce.Job: map    50% reduce 0%
18/12/05 16:46:44 INFO mapreduce.Job: map    100% reduce 0%
18/12/05 16:46:48 INFO mapreduce.Job: map    100% reduce 100%
18/12/05 16:46:50 INFO mapreduce.Job: Job    job_1543953844228_0025 completed suc
18/12/05 16:46:50 INFO mapreduce.Job: Counters: 49
    File System Counters
        FILE: Number of bytes   read=156411
        FILE: Number of bytes   written=813764
...
    Job Counters
```

```
Launched map tasks=2
Launched reduce tasks=1

...
Map-Reduce Framework
  Map input records=5260
  Map output records=25956
  Map output bytes=104493
  Map output materialized bytes=156417
  Input split bytes=346
  Combine input records=0
  Combine output records=0
  Reduce input groups=19
  Reduce shuffle bytes=156417
  Reduce input records=25956
  Reduce output records=19
  Spilled Records=51912
  Shuffled Maps =2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=193
  CPU time spent (ms)=4440
  Physical memory (bytes) snapshot=1942798336
  Virtual memory (bytes) snapshot=8463282176
  Total committed heap usage (bytes)=3177185280

...
18/12/05 16:46:50 INFO streaming.StreamJob: Output    directory: /example/wordlen
```

Viewing the Word Counts

Hadoop MapReduce saves its output into HDFS, so to see the actual word counts you must look at the file in HDFS within the cluster by executing the following command:

[click here to view code image](#)

```
hdfs dfs -text /example/wordlengthsoutput/part-00000
```

Here are the results of the preceding command:

[click here to view code image](#)

```
8/12/05 16:47:19 INFO lzo.GPLNativeCodeLoader: Loaded    native gpl library
18/12/05 16:47:19 INFO lzo.LzoCodec: Successfully loaded    & initialized native-
1      1140
2      3869
3      4699
4      5651
5      3668
6      2719
7      1624
8      1062
9       855
10     317
11     189
12     95
13     35
14     13
15     9
16     6
17     3
18     1
23     1
```

Deleting Your Cluster So You Do Not Incur Charges

Caution: Be sure to delete your cluster(s) and associated resources (like storage) so you don't incur additional charges. In the Azure portal, click **All resources** to see your list of resources, which will include the cluster you set up and the storage account you set up. Both can incur charges if you do not delete them. Select each resource and click the **Delete** button to remove it. You'll be asked to confirm by typing `yes`. For more information, see:

```
https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po
```

6.6 SPARK

In this section, we'll overview Apache Spark. We'll use the Python **PySpark library** and Spark's functional-style filter/map/reduce capabilities to implement a simple word count example that summarizes the word counts in *Romeo and Juliet*.

16.6.1 Spark Overview

When you process truly big data, performance is crucial. Hadoop is geared to disk-based batch processing—reading the data from disk, processing the data and writing the results back to disk. Many big-data applications demand better performance than is possible with disk-intensive operations. In particular, fast streaming applications that require either real-time or near-real-time processing won't work in a disk-based architecture.

History

Spark was initially developed in 2009 at U. C. Berkeley and funded by DARPA (the Defense Advanced Research Projects Agency). Initially, it was created as a distributed execution engine for high-performance machine learning.⁸ It uses an **in-memory architecture** that “has been used to sort 100 TB of data 3X faster than Hadoop MapReduce on 1/10th of the machines”⁹ and runs some workloads up to 100 times faster than Hadoop.⁰ Spark's significantly better performance on batch-processing tasks is leading many companies to replace Hadoop MapReduce with Spark.^{1, 2 3}

⁸ <https://gigaom.com/2014/06/28/4-reasons-why-spark-could-jolt-hadoop-into-hyperdrive/>.

⁹ <https://spark.apache.org/faq.html>.

⁰ <https://spark.apache.org/>.

¹ <https://bigdata-madesimple.com/is-spark-better-than-hadoop-map-reduce/>.

² <https://www.datanami.com/2018/10/18/is-hadoop-officially-dead/>.

³ <https://blog.thecodeteam.com/2018/01/09/changing-face-data-analytics-fast-data-displaces-big-data/>.

Architecture and Components

Though it was initially developed to run on Hadoop and use Hadoop components like HDFS and YARN, Spark can run standalone on a single computer (typically for learning and testing purposes), standalone on a cluster or using various cluster managers and distributed storage systems. For resource management, Spark runs on Hadoop YARN, Apache Mesos, Amazon EC2 and Kubernetes, and it supports many distributed storage systems, including HDFS, Apache Cassandra, Apache HBase and Apache Hive. ⁴

⁴ <http://spark.apache.org/>.

At the core of Spark are **resilient distributed datasets (RDDs)**, which you'll use to process distributed data using functional-style programming. In addition to reading data from disk and writing data to disk, Hadoop uses replication for fault tolerance, which adds even more disk-based overhead. RDDs eliminate this overhead by remaining in memory—using disk only if the data will not fit in memory—and by not replicating data. Spark handles fault tolerance by remembering the steps used to create each RDD, so it can rebuild a given RDD if a cluster node fails. ⁵

⁵ <https://spark.apache.org/research.html>.

Spark distributes the operations you specify in Python to the cluster's nodes for parallel execution. Spark streaming enables you to process data as it's received. Spark `DataFrames`, which are similar to pandas `DataFrames`, enable you to view RDDs as a collection of named columns. You can use Spark `DataFrames` with Spark SQL to perform queries on distributed data. Spark also includes **Spark MLlib** (the Spark Machine Learning Library), which enables you to perform machine-learning algorithms, like those you learned in [chapters 14 and 15](#). We'll use RDDs, Spark streaming, `DataFrames` and Spark SQL in the next few examples.

Providers

Hadoop providers typically also provide Spark support. In addition to the providers listed in [section 16.5](#), there are Spark-specific vendors like Databricks. They provide a “zero-management cloud platform built around Spark.” ⁶ Their website also is an excellent resource for learning Spark. The paid Databricks platform runs on Amazon AWS or Microsoft Azure. Databricks also provides a free Databricks Community Edition, which is a great way to get started with both Spark and the Databricks environment.

⁶ <https://databricks.com/product/faq>.

16.6.2 Docker and the Jupyter Docker Stacks

In this section, we'll show how to download and execute a Docker stack containing Spark and the PySpark module for accessing Spark from Python. You'll write the Spark example's code in a Jupyter Notebook. First, let's overview Docker.

Docker

Docker is a tool for packaging software into **containers** (also called **images**) that bundle *everything* required to execute that software across platforms. Some software packages we use in this chapter require complicated setup and configuration. For many of these, there are

preexisting Docker containers that you can download for free and execute locally on your desktop or notebook computers. This makes Docker a great way to help you get started with new technologies quickly and conveniently.

Docker also helps with *reproducibility* in research and analytics studies. You can create custom Docker containers that are configured with the versions of every piece of software and every library you used in your study. This would enable others to recreate the environment you used, then reproduce your work, and will help you reproduce your results at a later time. We'll use Docker in this section to download and execute a Docker container that's preconfigured to run Spark applications.

Installing Docker

You can install Docker for Windows 10 Pro or macOS at:

```
https://www.docker.com/products/docker-desktop
```

On Windows 10 Pro, you must allow the "Docker for Windows.exe" installer to make changes to your system to complete the installation process. To do so, click **Yes** when Windows asks if you want to allow the installer to make changes to your system.⁷ Windows 10 Home users must use Virtual Box as described at:

⁷Some Windows users might have to follow the instructions under **Allow specific apps to make changes to controlled folders** at <https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-exploit-guard/customize-controlled-folders-exploit-guard>.

```
https://docs.docker.com/machine/drivers/virtualbox/
```

Linux users should install Docker Community Edition as described at:

```
https://docs.docker.com/install/overview/
```

For a general overview of Docker, read the **Getting started** guide at:

```
https://docs.docker.com/get-started/
```

Jupyter Docker Stacks

The Jupyter Notebooks team has preconfigured several Jupyter "Docker stacks" containers for common Python development scenarios. Each enables you to use Jupyter Notebooks to experiment with powerful capabilities without having to worry about complex software setup issues. In each case, you can open JupyterLab in your web browser, open a notebook in JupyterLab and start coding. JupyterLab also provides a **Terminal window** that you can use in your browser like your computer's Terminal, Anaconda Command Prompt or shell. Everything we've shown you in IPython to this point can be executed using IPython in JupyterLab's Terminal window.

We'll use the `jupyter/pyspark-notebook` Docker stack, which is preconfigured with

everything you need to create and test Apache Spark apps on your computer. When combined with installing other Python libraries we've used throughout the book, you can implement most of this book's examples using this container. For more about the available Docker stacks, visit:

```
https://jupyter-docker-stacks.readthedocs.io/en/latest/index.html
```

Run Jupyter Docker Stack

Before performing the next step, ensure that JupyterLab is not currently running on your computer. Let's download and run the `jupyter/pyspark-notebook` Docker stack. To ensure that you do not lose your work when you close the Docker container, we'll attach a local file-system folder to the container and use it to save your notebook—Windows users should replace `\` with `^.`:

[lick here to view code image](#)

```
docker run -p 8888:8888 -p 4040:4040 -it --user root \
-v fullPathToTheFolderYouWantToUse:/home/jovyan/work \
jupyter/pyspark-notebook:14fd9bf9cfc1 start.sh jupyter lab
```

The first time you run the preceding command, Docker will download the Docker container named:

[lick here to view code image](#)

```
jupyter/pyspark-notebook:14fd9bf9cfc1
```

The notation "`:14fd9bf9cfc1`" indicates the specific `jupyter/pyspark-notebook` container to download. At the time of this writing, `14fd9bf9cfc1` was the newest version of the container. Specifying the version as we did here helps with *reproducibility*. Without the "`:14fd9bf9cfc1`" in the command, Docker will download the *latest* version of the container, which might contain different software versions and might not be compatible with the code you're trying to execute. The Docker container is nearly 6GB, so the initial download time will depend on your Internet connection's speed.

Opening JupyterLab in Your Browser

Once the container is downloaded and running, you'll see a statement in your Command Prompt, Terminal or shell window like:

[lick here to view code image](#)

```
Copy/paste this URL into your browser when you connect for the first time, to l
```

```
http://(bb00eb337630 or 127.0.0.1):8888/?token=
9570295e90ee94ecef75568b95545b7910a8f5502e6f5680
```

Copy the long hexadecimal string (the string on your system will differ from this one):

```
9570295e90ee94ecef75568b95545b7910a8f5502e6f5680
```

then open `http://localhost:8888/lab` in your browser (localhost corresponds to `127.0.0.1` in the preceding output) and *paste* your token in the **Password or token** field. Click **Log in** to be taken to the JupyterLab interface. If you accidentally close your browser, go to `http://localhost:8888/lab` to continue your session.

When running in this Docker container, the `work` folder in the **Files** tab at the left side of JupyterLab represents the folder you attached to the container in the `docker run` command's `-v` option. From here, you can open the notebook files we provide for you. Any new notebooks or other files you create will be saved to this folder by default. Because the Docker container's `work` folder is connected to a folder on your computer, any files you create in JupyterLab will remain on your computer, even if you decide to delete the Docker container.

Accessing the Docker Container's Command Line

Each Docker container has a command-line interface like the one you've used to run IPython throughout this book. Via this interface, you can install Python packages into the Docker container and even use IPython as you've done previously.

Open a separate Anaconda Command Prompt, Terminal or shell and list the currently running Docker containers with the command:

```
docker ps
```

The output of this command is wide, so the lines of text will likely wrap, as in:

[lick here to view code image](#)

CONTAINER ID	IMAGE	STATUS	PORTS	COMMAND
CREATED				
NAMES				
f54f62b7e6d5	jupyter/pyspark-notebook:14fd9bf9cfc1	Up 2 minutes	0.0.0.0:8888->8888/tcp	"tini -g -- /bin/bash"
friendly_pascal				

In the last line of our system's output under the column head `NAMES` in the third line is the name that Docker randomly assigned to the running container—`friendly_pascal`—the name on your system will differ. To access the container's command line, execute the following command, replacing `container_name` with the running container's name:

```
docker exec -it container_name /bin/bash
```

The Docker container uses Linux under the hood, so you'll see a Linux prompt where you can enter commands.

The app in this section will use features of the NLTK and TextBlob libraries you used in the “Natural Language Processing” chapter. Neither is preinstalled in the Jupyter Docker stacks. To install NLTK and TextBlob enter the command:

```
conda install -c conda-forge nltk textblob
```

Stopping and Restarting a Docker Container

Every time you start a container with `docker run`, Docker gives you a new instance that does *not* contain any libraries you installed previously. For this reason, you should keep track of your container name, so you can use it from another Anaconda Command Prompt, Terminal or shell window to stop the container and restart it. The command

```
docker stop container_name
```

will shut down the container. The command

```
docker restart container_name
```

will restart the container. Docker also provides a GUI app called Kitematic that you can use to manage your containers, including stopping and restarting them. You can get the app from <https://kitematic.com/> and access it through the Docker menu. The following user guide overviews how to manage containers with the tool:

```
https://docs.docker.com/kitematic/userguide/
```

16.6.3 Word Count with Spark

In this section, we'll use Spark's filtering, mapping and reducing capabilities to implement a simple word count example that summarizes the words in *Romeo and Juliet*. You can work with the existing notebook named `RomeoAndJulietCounter.ipynb` in the `SparkWordCount` folder (into which you should copy your `RomeoAndJuliet.txt` file from the “ Natural Language Processing” chapter), or you can create a new notebook, then enter and execute the snippets we show.

Loading the NLTK Stop Words

In this app, we'll use techniques you learned in the “ Natural Language Processing” chapter to eliminate stop words from the text before counting the words' frequencies. First, download the NLTK stop words:

[click here to view code image](#)

```
[1]: import nltk
    nltk.download('stopwords')
[nltk_data] Downloading package stopwords to /home/jovyan/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[1]: True
```

Next, load the stop words:

[click here to view code image](#)

```
[2]: from nltk.corpus import stopwords
```

```
stop_words = stopwords.words('english')
```

Configuring a SparkContext

A **SparkContext** (from module `pyspark`) object gives you access to Spark's capabilities in Python. Many Spark environments create the `SparkContext` for you, but in the Jupyter `pyspark-notebook` Docker stack, you must create this object.

First, let's specify the configuration options by creating a **SparkConf** object (from module `pyspark`). The following snippet calls the object's `setAppName` method to specify the Spark application's name and calls the object's `setMaster` method to specify the Spark cluster's URL. The URL `'local[*]'` indicates that Spark is executing on your `local` computer (as opposed to a cloud-based cluster), and the asterisk indicates that Spark should run our code using the same number of *threads* as there are cores on the computer:

[lick here to view code image](#)

```
[3]: from pyspark import SparkConf
     configuration = SparkConf().setAppName('RomeoAndJulietCounter') \
         .setMaster('local[*]')
```

Threads enable a single node cluster to execute portions of the Spark tasks *concurrently* to simulate the parallelism that Spark clusters provide. When we say that two tasks are operating concurrently, we mean that they're both making progress at once—typically by executing a task for a short burst of time, then allowing another task to execute. When we say that two tasks are operating in *parallel*, we mean that they're executing simultaneously, which is one of the key benefits of Hadoop and Spark executing on cloud-based clusters of computers.

Next, create the `SparkContext`, passing the `SparkConf` as its argument:

[lick here to view code image](#)

```
[4]: from pyspark import SparkContext
     sc = SparkContext(conf=configuration)
```

Reading the Text File and Mapping It to Words

You work with a `SparkContext` using functional-style programming techniques, like filtering, mapping and reduction, applied to a **resilient distributed dataset (RDD)**. An RDD takes data stored throughout a cluster in the Hadoop file system and enables you to specify a series of processing steps to transform the data in the RDD. These processing steps are *lazy* (chapter 5)—they do not perform any work until you indicate that Spark should process the task.

The following snippet specifies three steps:

- `SparkContext` method `textFile` loads the lines of text from `RomeoAndJuliet.txt` and returns it as an **RDD** (from module `pyspark`) of strings that represent each line.
- RDD method `map` uses its `lambda` argument to remove all punctuation with `TextBlob`'s

`strip_punc` function and to convert each line of text to lowercase. This method returns a new RDD on which you can specify additional tasks to perform.

- RDD method **flatMap** uses its `lambda` argument to map each line of text into its words and produces a single list of words, rather than the individual lines of text. The result of `flatMap` is a new RDD representing all the words in *Romeo and Juliet*.

[lick here to view code image](#)

```
[5]: from textblob.utils import strip_punc
    tokenized = sc.textFile('RomeoAndJuliet.txt')\
        .map(lambda line: strip_punc(line, all=True).lower())\
        .flatMap(lambda line: line.split())
```

Removing the Stop Words

Next, let's use RDD method **filter** to create a new RDD with no stop words remaining:

[lick here to view code image](#)

```
[6]: filtered = tokenized.filter(lambda word: word not in stop_words)
```

Counting Each Remaining Word

Now that we have only the non-stop-words, we can count the number of occurrences of each word. To do so, we first `map` each word to a tuple containing the word and a count of 1. This is similar to what we did in Hadoop MapReduce. Spark will distribute the reduction task across the cluster's nodes. On the resulting RDD, we then call the method **reduceByKey**, passing the `operator` module's `add` function as an argument. This tells method `reduceByKey` to *add* the counts for tuples that contain the same word (the key):

[lick here to view code image](#)

```
[7]: from operator import add
    word_counts = filtered.map(lambda word: (word, 1)).reduceByKey(add)
```

Locating Words with Counts Greater Than or Equal to 60

Since there are hundreds of words in *Romeo and Juliet*, let's filter the RDD to keep only those words with 60 or more occurrences:

[lick here to view code image](#)

```
[8]: filtered_counts = word_counts.filter(lambda item: item[1] >= 60)
```

Sorting and Displaying the Results

At this point, we've specified all the steps to count the words. When you call RDD method **collect**, Spark initiates all the processing steps we specified above and returns a list containing the final results—in this case, the tuples of words and their counts. From your perspective, everything appears to execute on one computer. However, if the `SparkContext`

is configured to use a cluster, Spark will divide the tasks among the cluster's worker nodes for you. In the following snippet, sort in descending order (`reverse=True`) the list of tuples by their counts (`itemgetter(1)`).

The following snippet calls method `collect` to obtain the results and sorts those results in descending order by word count:

[lick here to view code image](#)

```
[9]: from operator import itemgetter
      sorted_items = sorted(filtered_counts.collect(),
                           key=itemgetter(1), reverse=True)
```

Finally, let's display the results. First, we determine the word with the most letters so we can right-align all the words in a field of that length, then we display each word and its count:

[lick here to view code image](#)

```
[10]: max_len = max([len(word) for word, count in sorted_items])
      for word, count in sorted_items:
          print(f'{word:>{max_len}}: {count}')
[10]: romeo: 298
      thou: 277
      juliet: 178
      thy: 1708
      nurse: 146
      capulet: 141
      love: 136
      thee: 135
      shall: 110
      lady: 109
      friar: 104
      come: 94
      mercutio: 83
      good: 80
      benvolio: 79
      enter: 75
      go: 75
      i'll: 71
      tybalt: 69
      death: 69
      night: 68
      lawrence: 67
      man: 65
      hath: 64
      one: 60
```

16.6.4 Spark Word Count on Microsoft Azure

As we said previously, we want to expose you to both tools you can use for free and real-world development scenarios. In this section, you'll implement the Spark word-count example on a Microsoft Azure HDInsight Spark cluster.

Create an Apache Spark Cluster in HDInsight Using the Azure Portal

The following link explains how to set up a Spark cluster using the HDInsight service:

<https://docs.microsoft.com/en-us/azure/hdinsight/spark/apache-spark-jupyter-spark>

While following the **Create an HDInsight Spark cluster** steps, note the same issues we listed in the Hadoop cluster setup earlier in this chapter and for the **Cluster type** select **Spark**.

Again, the default cluster configuration provides more resources than you need for our examples. So, in the **Cluster summary**, perform the steps shown in the Hadoop cluster setup to change the number of worker nodes to 2 and to configure the worker and head nodes to use **D3 v2** computers. When you click **Create**, it takes 20 to 30 minutes to configure and deploy your cluster.

Install Libraries into a Cluster

If your Spark code requires libraries that are not installed in the HDInsight cluster, you'll need to install them. To see what libraries are installed by default, you can use ssh to log into your cluster (as we showed earlier in the chapter) and execute the command:

```
/usr/bin/anaconda/envs/py35/bin/conda list
```

Since your code will execute on multiple cluster nodes, libraries must be installed on *every* node. Azure requires you to create a Linux shell script that specifies the commands to install the libraries. When you submit that script to Azure, it validates the script, then executes it on every node. Linux shell scripts are beyond this book's scope, and the script must be hosted on a web server from which Azure can download the file. So, we created an install script for you that installs the libraries we use in the Spark examples. Perform the following steps to install these libraries:

1. In the Azure portal, select your cluster.
2. In the list of items under the cluster's search box, click **Script Actions**.
3. Click **Submit new** to configure the options for the library installation script. For the **Script type** select **Custom**, for the **Name** specify `libraries` and for the **Bash script URI** use:

```
http://deitel.com/bookresources/IntroToPython/install_libraries.sh
```
4. Check both **Head** and **Worker** to ensure that the script installs the libraries on all the nodes.
5. Click **Create**.

When the cluster finishes executing the script, if it executed successfully, you'll see a green check next to the script name in the list of script actions. Otherwise, Azure will notify you that there were errors.

Copying `RomeoAndJuliet.txt` to the HDInsight Cluster

As you did in the Hadoop demo, let's use the `scp` command to upload to the cluster the `RomeoAndJuliet.txt` file you used in the "Natural Language Processing" chapter. In a Command Prompt, Terminal or shell, change to the folder containing the file (we assume this

chapter's `ch16` folder), then enter the following command. Replace *YourClusterName* with the name you specified when creating your cluster and press *Enter* only when you've typed the entire command. The colon is required and indicates that you'll supply your cluster password when prompted. At that prompt, type the password you specified when setting up the cluster, then press *Enter*:

```
scp RomeoAndJuliet.txt sshuser@YourClusterName-ssh.azurehdinsight.net:
```

Next, use `ssh` to log into your cluster and access its command line. In a Command Prompt, Terminal or shell, execute the following command. Be sure to replace *YourClusterName* with your cluster name. Again, you'll be prompted for your cluster password:

```
ssh sshuser@YourClusterName-ssh.azurehdinsight.net
```

To work with the `RomeoAndJuliet.txt` file in Spark, first use the `ssh` session to copy the file into the cluster's Hadoop's file system by executing the following command. Once again, we'll use the already existing folder `/examples/data` that Microsoft includes for use with HDInsight tutorials. Again, press *Enter* only when you've typed the entire command:

```
hadoop fs -copyFromLocal RomeoAndJuliet.txt
/example/data/RomeoAndJuliet.txt
```

Accessing Jupyter Notebooks in HDInsight

At the time of this writing, HDInsight uses the *old* Jupyter Notebook interface, rather than the newer JupyterLab interface shown earlier. For a quick overview of the old interface see:

```
https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Notebook%20
```



o access Jupyter Notebooks in HDInsight, in the Azure portal select **All resources**, then your cluster. In the **Overview** tab, select **Jupyter notebook** under **Cluster dashboards**. This opens a web browser window and asks you to log in. Use the username and password you specified when setting up the cluster. If you did not specify a username, the default is `admin`. Once you log in, Jupyter displays a folder containing `PySpark` and `Scala` subfolders. These contain Python and Scala Spark tutorials.

Uploading the `RomeoAndJulietCounter.ipynb` Notebook

You can create new notebooks by clicking **New** and selecting `PySpark3`, or you can upload existing notebooks from your computer. For this example, let's upload the previous section's `RomeoAndJulietCounter.ipynb` notebook and modify it to work with Azure. To do so, click the **Upload** button, navigate to the `ch16` example folder's `SparkWordCount` folder, select `RomeoAndJulietCounter.ipynb` and click **Open**. This displays the file in the folder with an **Upload** button to its right. Click that button to place the notebook in the current folder. Next, click the notebook's name to open it in a new browser tab. Jupyter will display a **Kernel not found** dialog. Select **PySpark3** and click **OK**. Do not run any cells yet.

Modifying the Notebook to Work with Azure

Perform the following steps, executing each cell as you complete the step:

1. The HDInsight cluster will not allow NLTK to store the downloaded stop words in NLTK's default folder because it's part of the system's protected folders. In the first cell, modify the call `nltk.download('stopwords')` as follows to store the stop words in the current folder (`'.'`):

```
nltk.download('stopwords', download_dir='.')
```

When you execute the first cell, Starting Spark application appears below the cell while HDInsight sets up a `SparkContext` object named `sc` for you. When this task is complete, the cell's code executes and downloads the stop words.

2. In the second cell, before loading the stop words, you must tell NLTK that they're located in the current folder. Add the following statement after the `import` statement to tell NLTK to search for its data in the current folder:

```
nltk.data.path.append('.')
```

3. Because HDInsight sets up the `SparkContext` object for you, the third and fourth cells of the original notebook are not needed, so you can delete them. To do so, either click inside it and select **Delete Cells** from Jupyter's **Edit** menu, or click in the white margin to the cell's left and type `dd`.
4. In the next cell, specify the location of `RomeoAndJuliet.txt` in the underlying Hadoop file system. Replace the string `'RomeoAndJuliet.txt'` with the string

```
'wasb:///example/data/RomeoAndJuliet.txt'
```

The notation `wasb:///` indicates that `RomeoAndJuliet.txt` is stored in a Windows Azure Storage Blob (WASB)—Azure's interface to the HDFS file system.

5. Because Azure currently uses Python 3.5.x, it does not support f-strings. So, in the last cell, replace the f-string with the following older-style Python string formatting using the string method `format`:

```
print('{:>{width}}: {}'.format(word, count, width=max_len))
```

You'll see the same final results as in the previous section.

Caution: Be sure to delete your cluster and other resources when you're done with them, so you do not incur charges. For more information, see:

<https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po>

ote that when you delete your Azure resources, *your notebooks will be deleted as well*. You

can download the notebook you just executed by selecting **File > Download as > Notebook (.ipynb)** in Jupyter.

16.7 SPARK STREAMING: COUNTING TWITTER HASHTAGS USING THE PYSPARK-NOTEBOOK DOCKER STACK

In this section, you'll create and run a Spark streaming application in which you'll receive a stream of tweets on the topic(s) you specify and summarize the top-20 hashtags in a bar chart that updates every 10 seconds. For this purpose of this example, you'll use the Jupyter Docker container from the first Spark example.

There are two parts to this example. First, using the techniques from the “Data Mining Twitter” chapter, you'll create a script that streams tweets from Twitter. Then, we'll use Spark streaming in a Jupyter Notebook to read the tweets and summarize the hashtags.

The two parts will communicate with one another via networking **sockets**—a low-level view of *client/server networking* in which a *client* app communicates with a *server* app over a network using techniques similar to file I/O. A program can read from a socket or write to a socket similarly to reading from a file or writing to a file. The socket represents one endpoint of a connection. In this case, the *client* will be a Spark application, and the *server* will be a script that receives streaming tweets and sends them to the Spark app.

Launching the Docker Container and Installing Tweepy

For this example, you'll install the Tweepy library into the Jupyter Docker container. Follow section 16.6.2's instructions for launching the container and installing Python libraries into it. Use the following command to install Tweepy:

```
pip install tweepy
```

16.7.1 Streaming Tweets to a Socket

The script `starttweetstream.py` contains a modified version of the `TweetListener` class from the “Data Mining Twitter” chapter. It streams the specified number of tweets and sends them to a socket on the local computer. When the tweet limit is reached, the script closes the socket. You've already used Twitter streaming, so we'll focus only on what's new. Ensure that the file `keys.py` (in the `ch16` folder's `SparkHashtagSummarizer` subfolder) contains your Twitter credentials.

Executing the Script in the Docker Container

In this example, you'll use JupyterLab's Terminal window to execute `starttweetstream.py` in one tab, then use a notebook to perform the Spark task in another tab. With the Jupyter `pyspark-notebook` Docker container running, open

```
http://localhost:8888/lab
```

in your web browser. In JupyterLab, select **File > New > Terminal** to open a new tab containing a **Terminal**. This is a Linux-based command line. Typing the `ls` command and pressing *Enter* lists the current folder's contents. By default, you'll see the container's `work`

folder.

To execute `starttweetstream.py`, you must first navigate to the `SparkHashtagSummarizer` folder with the command ⁸:

⁸Windows users should note that Linux uses `/` rather than `\` to separate folders and that file and folder names are case sensitive.

```
cd work/SparkHashtagSummarizer
```

You can now execute the script with the command of the form

```
ipython starttweetstream.py number_of_tweets search_terms
```

where *number_of_tweets* specifies the total number of tweets to process and *search_terms* one or more space-separated strings to use for filtering tweets. For example, the following command would stream 1000 tweets about football:

```
ipython starttweetstream.py 1000 football
```

At this point, the script will display "Waiting for connection" and will wait until Spark connects to begin streaming the tweets.

`starttweetstream.py` import Statements

For discussion purposes, we've divided `starttweetstream.py` into pieces. First, we import the modules used in the script. The Python Standard Library's **socket module** provides the capabilities that enable Python apps to communicate via sockets.

[lick here to view code image](#)

```
1 # starttweetstream.py
2 """Script to get tweets on topic(s) specified as script argument(s)
3    and send tweet text to a socket for processing by Spark."""
4 import keys
5 import socket
6 import sys
7 import tweepy
8
```

Class `TweetListener`

Once again, you've seen most of the code in class `TweetListener`, so we focus only on what's new here:

- Method `__init__` (lines 12–17) now receives a `connection` parameter representing the socket and stores it in the `self.connection` attribute. We use this socket to send the hashtags to the Spark application.
- In method `on_status` (lines 24–44), lines 27–32 extract the hashtags from the Tweepy `Status` object, convert them to lowercase and create a space-separated string of the

hashtags to send to Spark. The key statement is line 39:

```
self.connection.send(hashtags_string.encode('utf-8'))
```

which uses the `connection` object's `send` method to send the tweet text to whatever application is reading from that socket. Method `send` expects as its argument a sequence of bytes. The string method call `encode('utf-8')` converts the string to bytes. Spark will automatically read the bytes and reconstruct the strings.

[lick here to view code image](#)

```
9 class TweetListener(tweepy.StreamListener):
10     """Handles incoming Tweet stream."""
11
12     def __init__(self, api, connection, limit=10000):
13         """Create instance variables for tracking number of tweets."""
14         self.connection = connection
15         self.tweet_count = 0
16         self.TWEET_LIMIT = limit # 10,000 by default
17         super().__init__(api) # call superclass's init
18
19     def on_connect(self):
20         """Called when your connection attempt is successful, enabling
21         you to perform appropriate application tasks at that point."""
22         print('Successfully connected to Twitter\n')
23
24     def on_status(self, status):
25         """Called when Twitter pushes a new tweet to you."""
26         # get the hashtags
27         hashtags = []
28
29         for hashtag_dict in status.entities['hashtags']:
30             hashtags.append(hashtag_dict['text'].lower())
31
32         hashtags_string = ' '.join(hashtags) + '\n'
33         print(f'Screen name: {status.user.screen_name}:')
34         print(f'Hashtags: {hashtags_string}')
35         self.tweet_count += 1 # track number of tweets processed
36
37         try:
38             # send requires bytes, so encode the string in utf-8 format
39             self.connection.send(hashtags_string.encode('utf-8'))
40         except Exception as e:
41             print(f'Error: {e}')
42
43         # if TWEET_LIMIT is reached, return False to terminate streaming
44         return self.tweet_count != self.TWEET_LIMIT
45
46     def on_error(self, status):
47         print(status)
48         return True
49
```

Main Application

Lines 50–80 execute when you run the script. You've connected to Twitter to stream tweets previously, so here we discuss only what's new in this example.

Line 51 gets the number of tweets to process by converting the command-line argument

`sys.argv[1]` to an integer. Recall that element 0 represents the script's name.

[lick here to view code image](#)

```
50 if __name__ == '__main__':  
51     tweet_limit = int(sys.argv[1]) # get maximum number of tweets
```

Line 52 calls the `socket` module's **socket function**, which returns a socket object that we'll use to wait for a connection from the Spark application.

[lick here to view code image](#)

```
52     client_socket = socket.socket() # create a socket  
53
```

Line 55 calls the socket object's **bind method** with a tuple containing the hostname or IP address of the computer and the port number on that computer. Together these represent where this script will wait for an initial connection from another app:

[lick here to view code image](#)

```
54     # app will use localhost (this computer) port 9876  
55     client_socket.bind(('localhost', 9876))  
56
```

Line 58 calls the socket's **listen method**, which causes the script to *wait* until a connection is received. This is the statement that prevents the Twitter stream from starting until the Spark application connects.

[lick here to view code image](#)

```
57     print('Waiting for connection')  
58     client_socket.listen() # wait for client to connect  
59
```

Once the Spark application connects, line 61 calls socket method **accept**, which accepts the connection. This method returns a tuple containing a new socket object that the script will use to communicate with the Spark application and the IP address of the Spark application's computer.

[lick here to view code image](#)

```
60     # when connection received, get connection/client address  
61     connection, address = client_socket.accept()  
62     print(f'Connection received from {address}')
```

Next, we authenticate with Twitter and start the stream. Lines 73–74 set up the stream, passing the socket object `connection` to the `TweetListener` so that it can use the socket to send hashtags to the Spark application.

[lick here to view code image](#)

```
64     # configure Twitter access
65     auth = tweepy.OAuthHandler(keys.consumer_key, keys.consumer_secret)
66     auth.set_access_token(keys.access_token, keys.access_token_secret)
67
68     # configure Tweepy to wait if Twitter rate limits are reached
69     api = tweepy.API(auth, wait_on_rate_limit=True,
70                      wait_on_rate_limit_notify=True)
71
72     # create the Stream
73     twitter_stream = tweepy.Stream(api.auth,
74                                   TweetListener(api, connection, tweet_limit))
75
76     # sys.argv[2] is the first search term
77     twitter_stream.filter(track=sys.argv[2:])
78
```

Finally, lines 79–80 call the `close` method on the socket objects to release their resources.

[lick here to view code image](#)

```
79     connection.close()
80     client_socket.close()
```

16.7.2 Summarizing Tweet Hashtags; Introducing Spark SQL

In this section, you'll use Spark streaming to read the hashtags sent via a socket by the script `starttweetstream.py` and summarize the results. You can either create a new notebook and enter the code you see here or load the `hashtagsummarizer.ipynb` notebook we provide in the `ch16 examples` folder's `SparkHashtagSummarizer` subfolder.

Importing the Libraries

First, let's import the libraries used in this notebook. We'll explain the `pyspark` classes as we use them. From `IPython`, we imported the `display` module, which contains classes and utility functions that you can use in Jupyter. In particular, we'll use the `clear_output` function to remove an existing chart before displaying a new one:

[lick here to view code image](#)

```
[1]: from pyspark import SparkContext
    from pyspark.streaming import StreamingContext
    from pyspark.sql import Row, SparkSession
    from IPython import display
    import matplotlib.pyplot as plt
    import seaborn as sns
    %matplotlib inline
```

This Spark application summarizes hashtags in 10-second batches. After processing each batch, it displays a Seaborn barplot. The `IPython` magic

```
%matplotlib inline
```

indicates that Matplotlib-based graphics should be displayed in the notebook rather than in their own windows. Recall that Seaborn uses Matplotlib.

We've used several IPython magics throughout the book. There are many magics specifically for use in Jupyter Notebooks. For the complete list of magics see:

```
https://ipython.readthedocs.io/en/stable/interactive/magics.html
```

Utility Function to Get the `SparkSession`

As you'll soon see, you can use **Spark SQL** to query data in resilient distributed datasets (RDDs). Spark SQL uses a Spark **DataFrame** to get a table view of the underlying RDDs. A **SparkSession** (module `pyspark.sql`) is used to create a `DataFrame` from an RDD.

There can be only one `SparkSession` object per Spark application. The following function, which we borrowed from the *Spark Streaming Programming Guide*,⁹ defines the correct way to get a `SparkSession` instance if it already exists or to create one if it does not yet exist:⁰

⁹ <https://spark.apache.org/docs/latest/streaming-programming-guide.html#dataframe-and-sql-operations>.

⁰Because this function was borrowed from the *Spark Streaming Programming Guides DataFrame and SQL Operations* section

(<https://spark.apache.org/docs/latest/streaming-programming-guide.html#dataframe-and-sql-operations>), we did not rename it to use Python's standard function naming style, and we did not use single quotes to delimit strings.

[lick here to view code image](#)

```
[2]: def getSparkSessionInstance(sparkConf):
    """Spark Streaming Programming Guide's recommended method
    for getting an existing SparkSession or creating a new one."""
    if ("sparkSessionSingletonInstance" not in globals()):
        globals()["sparkSessionSingletonInstance"] = SparkSession \
            .builder \
            .config(conf=sparkConf) \
            .getOrCreate()
    return globals()["sparkSessionSingletonInstance"]
```

Utility Function to Display a Barchart Based on a Spark DataFrame

We call function `display_barplot` after Spark processes each batch of hashtags. Each call clears the previous Seaborn barplot, then displays a new one based on the Spark `DataFrame` it receives. First, we call the Spark `DataFrame`'s `toPandas` method to convert it to a pandas `DataFrame` for use with Seaborn. Next, we call the `clear_output` function from the `IPython.display` module. The keyword argument `wait=True` indicates that the function should remove the prior graph (if there is one), but only once the new graph is ready to display. The rest of the code in the function uses standard Seaborn techniques we've shown previously. The function call `sns.color_palette('cool', 20)` selects twenty colors from the Matplotlib 'cool' color palette:

[lick here to view code image](#)

```
[3]: def display_barplot(spark_df, x, y, time, scale=2.0, size=(16, 9)):  
    """Displays a Spark DataFrame's contents as a bar plot."""  
    df = spark_df.toPandas()  
  
    # remove prior graph when new one is ready to display  
    display.clear_output(wait=True)  
    print(f'TIME: {time}')  
  
    # create and configure a Figure containing a Seaborn barplot  
    plt.figure(figsize=size)  
    sns.set(font_scale=scale)  
    barplot = sns.barplot(data=df, x=x, y=y,  
                           palette=sns.color_palette('cool', 20))  
  
    # rotate the x-axis labels 90 degrees for readability  
    for item in barplot.get_xticklabels():  
        item.set_rotation(90)  
  
    plt.tight_layout()  
    plt.show()
```

Utility Function to Summarize the Top-20 Hashtags So Far

In Spark streaming, a **DStream** is a sequence of RDDs each representing a mini-batch of data to process. As you'll soon see, you can specify a function that is called to perform a task for every RDD in the stream. In this app, the function `count_tags` will summarize the hashtag counts in a given RDD, add them to the current totals (maintained by the `SparkSession`), then display an updated top-20 barplot so that we can see how the top-20 hashtags are changing over time.¹ For discussion purposes, we've broken this function into smaller pieces. First, we get the `SparkSession` by calling the utility function `getSparkSessionInstance` with the `SparkContext`'s configuration information. Every RDD has access to the `SparkContext` via the `context` attribute:

¹When this function gets called the first time, you might see an exceptions error message display if no tweets with hashtags have been received yet. This is because we simply display the error message in the standard output. That message will disappear as soon as there are tweets with hashtags.

[lick here to view code image](#)

```
[4]: def count_tags(time, rdd):  
    """Count hashtags and display top-20 in descending order."""  
    try:  
        # get SparkSession  
        spark = getSparkSessionInstance(rdd.context.getConf())
```

Next, we call the RDD's `map` method to map the data in the RDD to **Row** objects (from the `pyspark.sql` package). The RDDs in this example contain tuples of hashtags and counts. The `Row` constructor uses the names of its keyword arguments to specify the column names for each value in that row. In this case, `tag[0]` is the hashtag in the tuple, and `tag[1]` is the total count for that hashtag:

[lick here to view code image](#)

```
# map hashtag string-count tuples to Rows
rows = rdd.map(
    lambda tag: Row(hashtag=tag[0], total=tag[1]))
```

The next statement creates a Spark `DataFrame` containing the `Row` objects. We'll use this with Spark SQL to query the data to get the top-20 hashtags with their total counts:

[lick here to view code image](#)

```
# create a DataFrame from the Row objects
hashtags_df = spark.createDataFrame(rows)
```

To query a Spark `DataFrame`, first create a *table view*, which enables Spark SQL to query the `DataFrame` like a table in a relational database. Spark `DataFrame` method **`createOrReplaceTempView`** creates a temporary table view for the `DataFrame` and names the view for use in the `from` clause of a query:

[lick here to view code image](#)

```
# create a temporary table view for use with Spark SQL
hashtags_df.createOrReplaceTempView('hashtags')
```

Once you have a table view, you can query the data using Spark SQL. ² The following statement uses the `SparkSession` instance's `sql` method to perform a Spark SQL query that selects the `hashtag` and `total` columns from the `hashtags` table view, orders the selected rows by `total` in descending (`desc`) order, then returns the first 20 rows of the result (`limit 20`). Spark SQL returns a new Spark `DataFrame` containing the results:

² or details of Spark SQLs syntax, see <https://spark.apache.org/sql/>.

[lick here to view code image](#)

```
# use Spark SQL to get top 20 hashtags in descending order
top20_df = spark.sql(
    """select hashtag, total
    from hashtags
    order by total, hashtag desc
    limit 20""")
```

Finally, we pass the Spark `DataFrame` to our `display_barplot` utility function. The `hashtags` and `totals` will be displayed on the *x*- and *y*-axes, respectively. We also display the time at which `count_tags` was called:

[lick here to view code image](#)

```
display_barplot(top20_df, x='hashtag', y='total', time=time)
except Exception as e:
    print(f'Exception: {e}')
```

Getting the SparkContext

The rest of the code in this notebook sets up Spark streaming to read text from the `starttweetstream.py` script and specifies how to process the tweets. First, we create the `SparkContext` for connecting to the Spark cluster:

```
[5]: sc = SparkContext()
```

Getting the StreamingContext

For Spark streaming, you must create a `StreamingContext` (module `pyspark.streaming`), providing as arguments the `SparkContext` and how often in seconds to process batches of streaming data. In this app, we'll process batches every 10 seconds—this is the *batch interval*:

[lick here to view code image](#)

```
[6]: ssc = StreamingContext(sc, 10)
```

Depending on how fast data is arriving, you may wish to shorten or lengthen your batch intervals. For a discussion of this and other performance-related issues, see the Performance Tuning section of the *Spark Streaming Programming Guide*:

<https://spark.apache.org/docs/latest/streaming-programming-guide.html#performan>

Setting Up a Checkpoint for Maintaining State

By default, Spark streaming does not maintain state information as you process the stream of RDDs. However, you can use Spark **checkpointing** to keep track of the streaming state. Checkpointing enables:

- fault-tolerance for restarting a stream in cases of cluster node or Spark application failures, and
- stateful transformations, such as summarizing the data received so far—as we're doing in this example.

`StreamingContext` method **checkpoint** sets up the checkpointing folder:

[lick here to view code image](#)

```
[7]: ssc.checkpoint('hashtagsummarizer_checkpoint')
```

For a Spark streaming application in a cloud-based cluster, you'd specify a location within HDFS to store the checkpoint folder. We're running this example in the local Jupyter Docker image, so we simply specified the name of a folder, which Spark will create in the current folder (in our case, the `ch16` folder's `SparkHashtagSummarizer`). For more details on checkpointing, see

Connecting to the Stream via a Socket

`StreamingContext` method `socketTextStream` connects to a socket from which a stream of data will be received and returns a `DStream` that receives the data. The method's arguments are the hostname and port number to which the `StreamingContext` should connect—these must match where the `starttweetstream.py` script is waiting for the connection:

[lick here to view code image](#)

```
[8]: stream = ssc.socketTextStream('localhost', 9876)
```

Tokenizing the Lines of Hashtags

We use functional-style programming calls on a `DStream` to specify the processing steps to perform on the streaming data. The following call to `DStream`'s `flatMap` method tokenizes a line of space-separated hashtags and returns a new `DStream` representing the individual tags:

[lick here to view code image](#)

```
[9]: tokenized = stream.flatMap(lambda line: line.split())
```

Mapping the Hashtags to Tuples of Hashtag-Count Pairs

Next, similar to the Hadoop mapper earlier in this chapter, we use `DStream` method `map` to get a new `DStream` in which each hashtag is mapped to a hashtag-count pair (in this case as a tuple) in which the count is initially 1:

[lick here to view code image](#)

```
[10]: mapped = tokenized.map(lambda hashtag: (hashtag, 1))
```

Totaling the Hashtag Counts So Far

`DStream` method `updateStateByKey` receives a two-argument lambda that totals the counts for a given key and adds them to the prior total for that key:

[lick here to view code image](#)

```
[11]: hashtag_counts = tokenized.updateStateByKey(  
    lambda counts, prior_total: sum(counts) + (prior_total or 0))
```

Specifying the Method to Call for Every RDD

Finally, we use `DStream` method `foreachRDD` to specify that every processed RDD should be passed to function `count_tags`, which then summarizes the top-20 hashtags so far and

displays a barplot:

[lick here to view code image](#)

```
[12]: hashtag_counts.foreachRDD(count_tags)
```

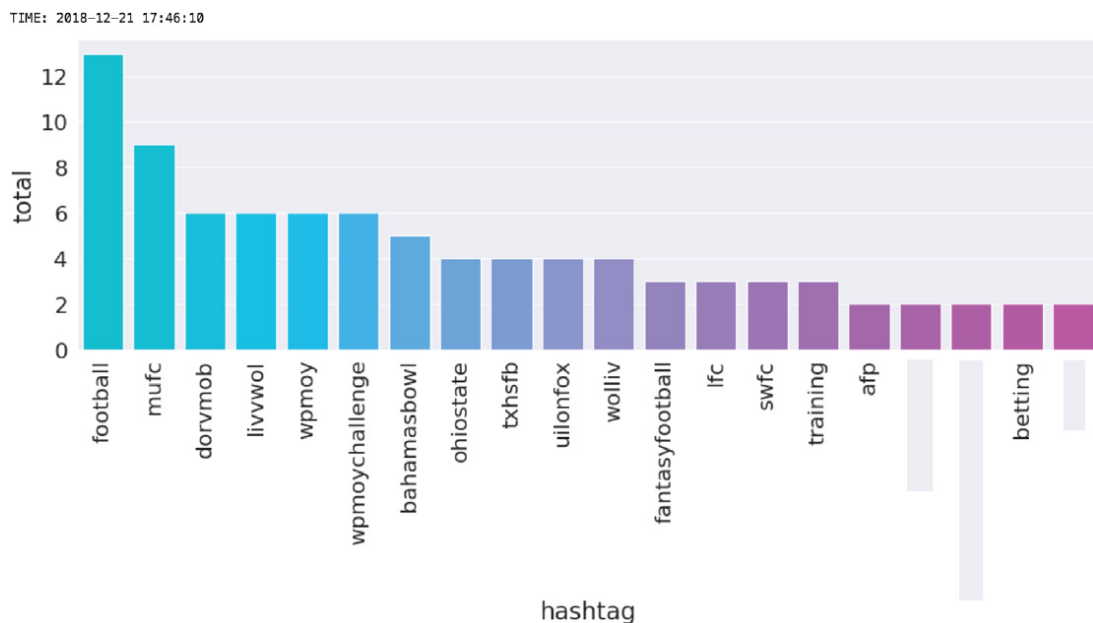
Starting the Spark Stream

Now, that we've specified the processing steps, we call the `StreamingContext`'s `start` method to connect to the socket and begin the streaming process.

[lick here to view code image](#)

```
[13]: ssc.start() # start the Spark streaming
```

The following shows a sample barplot produced while processing a stream of tweets about “football.” Because football is a different sport in the United States and the rest of the world the hashtags relate to both American football and what we call soccer—we grayed out three hashtags that were not appropriate for publication:



16.8 INTERNET OF THINGS AND DASHBOARDS

In the late 1960s, the Internet began as the ARPANET, which initially connected four universities and grew to 10 nodes by the end of 1970. ³ In the last 50 years, that has grown to billions of computers, smartphones, tablets and an enormous range of other device types connected to the Internet worldwide. *Any* device connected to the Internet is a “thing” in the Internet of Things (IoT).

³ <https://en.wikipedia.org/wiki/ARPANET#History..>

Each device has a unique Internet protocol address (IP address) that identifies it. The explosion of connected devices exhausted the approximately 4.3 billion available IPv4 (Internet Protocol version 4) addresses ⁴ and led to the development of IPv6, which supports approximately 3.4×10^{38} addresses (that's a lot of zeros). ⁵

⁴ https://en.wikipedia.org/wiki/IPv4_address_exhaustion.

⁵ <https://en.wikipedia.org/wiki/IPv6>.

“Top research firms such as Gartner and McKinsey predict a jump from the 6 billion connected devices we have worldwide today, to 20–30 billion by 2020.” ⁶ Various predictions say that number could be 50 billion. Computer-controlled, Internet-connected devices continue to proliferate. The following is a small subset IoT device types and applications.

⁶ <https://www.pubnub.com/developers/tech/how-pubnub-works/>.

IoT devices		
activity trackers—		
Apple Watch, FitBit,		
Amazon Dash ordering buttons		smart home—lights, garage openers, video cameras, doorbells, irrigation controllers, security devices, smart locks, smart plugs, smoke detectors, thermostats, air vents
Amazon Echo (Alexa), Apple HomePod (Siri), Google Home (Google Assistant)	healthcare—blood glucose monitors for diabetics, blood pressure monitors, electrocardiograms (EKG/ECG), electroencephalograms (EEG), heart monitors, ingestible sensors, pacemakers, sleep trackers,	tsunami sensors
	sensors—chemical, gas, GPS, humidity, light, motion, pressure, temperature,	tracking devices
appliances—ovens, coffee makers, refrigerators,		wine cellar refrigerators
driverless cars		wireless network devices
earthquake sensors		

though there's a lot of excitement and opportunity in IoT, not everything is positive. There are many security, privacy and ethical concerns. Unsecured IoT devices have been used to perform distributed-denial-of-service (DDOS) attacks on computer systems.⁷ Home security cameras that you intend to protect your home could potentially be hacked to allow others access to the video stream. Voice-controlled devices are always “listening” to hear their trigger words. This leads to privacy and security concerns. Children have accidentally ordered products on Amazon by talking to Alexa devices, and companies have created TV ads that would activate Google Home devices by speaking their trigger words and causing Google Assistant to read Wikipedia pages about a product to you.⁸ Some people worry that these devices could be used to eavesdrop. Just recently, a judge ordered Amazon to turn over Alexa recordings for use in a criminal case.⁹

⁷ <https://threatpost.com/iot-security-concerns-peeking-with-no-end-in-sight/131308/>.

⁸ <https://www.symantec.com/content/dam/symantec/docs/security-center/whitepapers/istr-security-voice-activated-smart-speakers-en.pdf>.

⁹ <https://techcrunch.com/2018/11/14/amazon-echo-recordings-judge-order-case/>.

This Section's Examples

In this section, we discuss the **publish/subscribe model** that IoT and other types of applications use to communicate. First, without writing any code, you'll build a web-based dashboard using Freeboard.io and subscribe to a sample live stream from the PubNub service. Next, you'll simulate an Internet-connected thermostat which publishes messages to the free Dweet.io service using the Python module Dweeepy, then create a dashboard visualization of it with Freeboard.io. Finally, you'll build a Python client that subscribes to a sample live stream from the PubNub service and dynamically visualizes the stream with Seaborn and a Matplotlib `FuncAnimation`.

16.8.1 Publish and Subscribe

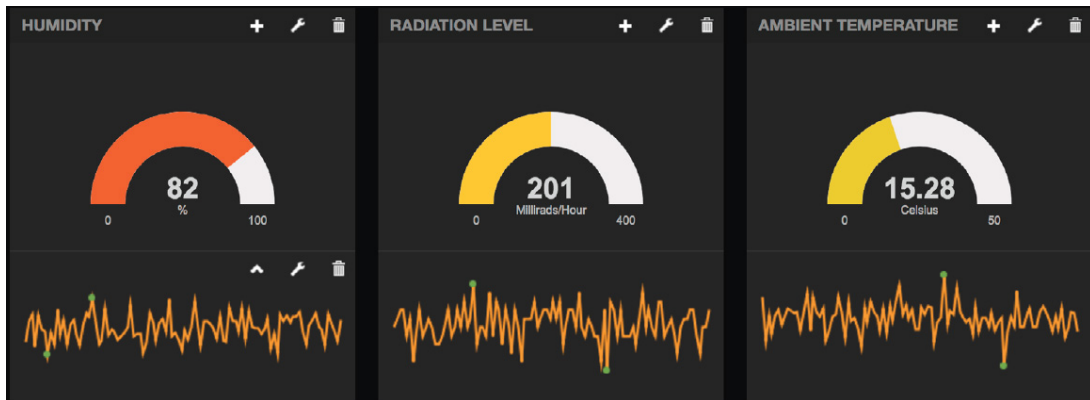
IoT devices (and many other types of devices and applications) commonly communicate with one another and with applications via **pub/sub (publisher/subscriber) systems**. A **publisher** is any device or application that sends a message to a cloud-based service, which in turn sends that message to all **subscribers**. Typically each publisher specifies a topic or channel, and each subscriber specifies one or more **topics** or **channels** for which they'd like to receive messages. There are many pub/sub systems in use today. In the remainder of this section, we'll use PubNub and Dweet.io. You also should investigate Apache Kafka—a Hadoop ecosystem component that provides a high-performance publish/subscribe service, real-time stream processing and storage of streamed data.

16.8.2 Visualizing a PubNub Sample Live Stream with a Freeboard Dashboard

PubNub is a pub/sub service geared to real-time applications in which any software and device connected to the Internet can communicate via small messages. Some of their common use-cases include IoT, chat, online multiplayer games, social apps and collaborative apps. PubNub provides several live streams for learning purposes, including one that simulates IoT sensors (Section 16.8.5 lists the others).

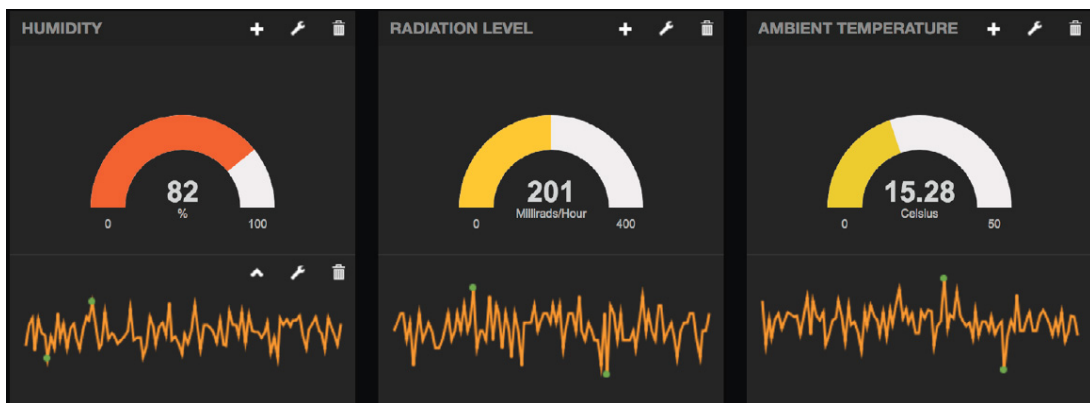
One common use of live data streams is visualizing them for monitoring purposes. In this section, you'll connect PubNub's live simulated sensor stream to a Freeboard.io web-based dashboard. A car's dashboard visualizes data from your car's sensors, showing information such as the outside temperature, your speed, engine temperature, the time and the amount of gas remaining. A web-based dashboard does the same thing for data from various sources, including IoT devices.

Freeboard.io is a cloud-based dynamic dashboard visualization tool. You'll see that, without writing any code, you can easily connect Freeboard.io to various data streams and visualize the data as it arrives. The following dashboard visualizes data from three of the four simulated sensors in the PubNub simulated IoT sensors stream:



For each sensor, we used a **Gauge** (the semicircular visualizations) and a **Sparkline** (the jagged lines) to visualize the data. When you complete this section, you'll see the **Gauges** and **Sparklines** frequently moving as new data arrives multiple times per second.

In addition to their paid service, Freeboard.io provides an open-source version (with fewer options) on GitHub. They also provide tutorials that show how to add *custom plug-ins*, so you can develop your own visualizations to add to their dashboards.



Signing up for Freeboard.io

For this example, register for a Freeboard.io 30-day trial at

```
https://freeboard.io/signup
```

Once you've registered, the **My Freeboards** page appears. If you'd like, you can click the **Try a Tutorial** button and visualize data from your smartphone.

Creating a New Dashboard

n the upper-right corner of the **My Freeboards** page, enter `Sensor Dashboard` in the **enter a name** field, then click the **Create New** button to create a dashboard. This displays the dashboard designer.

Adding a Data Source

If you add your data source(s) before designing your dashboard, you'll be able to configure each visualization as you add it:

1. Under **DATASOURCES**, click **ADD** to specify a new data source.
2. The **DATASOURCE** dialog's **TYPE** drop-down list shows the currently supported data sources, though you can develop plug-ins for new data sources as well. ^o Select **PubNub**. The web page for each PubNub sample live stream specifies the **Channel** and **Subscribe key**. Copy these values from PubNub's Sensor Network page at `https://www.pubnub.com/developers/realtime-data-streams/sensor-network/`, then insert their values in the corresponding **DATASOURCE** dialog fields. Provide a **NAME** for your data source, then click **SAVE**.

^o**Some of the listed data sources are available only via Freeboard.io, not the open source Freeboard on GitHub.**

Adding a Pane for the Humidity Sensor

A Freeboard.io dashboard is divided into panes that group visualizations. Multiple panes can be dragged to rearrange them. Click the + **Add Pane** button to add a new pane. Each pane can have a title. To set it, click the wrench icon on the pane, specify `Humidity` for the **TITLE**, then click **SAVE**.

Adding a Gauge to the Humidity Pane

A Freeboard.io dashboard is divided into panes that group visualizations. Multiple panes can be dragged to rearrange them. Click the + **Add Pane** button to add a new pane. Each pane can have a title. To set it, click the wrench icon on the pane, specify `Humidity` for the **TITLE**, then click **SAVE**.

Notice that the humidity value has four digits of precision to the right of the decimal point. PubNub supports JavaScript expressions, so you can use them to perform calculations or format data. For example, you can use JavaScript's function `Math.round` to round the humidity value to the closest integer. To do so, hover the mouse over the gauge and click its wrench icon. Then, insert `"Math.round("` before the text in the **VALUE** field and `")"` after the text, then click **SAVE**.

Adding a Sparkline to the Humidity Pane

A **sparkline** is a line graph without axes that's typically used to give you a sense of how a data value is changing over time. Add a sparkline for the humidity sensor by clicking the humidity pane's + button, then selecting **Sparkline** from the **TYPE** drop-down list. For the **VALUE**, once again select your data source and **humidity**, then click **SAVE**.

Completing the Dashboard

Using the techniques above, add two more panes and drag them to the right of the first.

Name them **Radiation Level** and **Ambient Temperature**, respectively, and configure each pane with a **Gauge** and **Sparkline** as shown above. For the **Radiation Level** gauge, specify `Millirads/Hour` for the **UNITS** and `400` for the **MAXIMUM**. For the **Ambient Temperature** gauge, specify `Celsius` for the **UNITS** and `50` for the **MAXIMUM**.

16.8.3 Simulating an Internet-Connected Thermostat in Python

Simulation is one of the most important applications of computers. We used simulation with dice rolling in earlier chapters. With IoT, it's common to use simulators to test your applications, especially when you do not have access to actual devices and sensors while developing applications. Many cloud vendors have IoT simulation capabilities, such as IBM Watson IoT Platform and IOTIFY.io.

Here, you'll create a script that simulates an Internet-connected thermostat publishing periodic JSON messages—called dweets—to `dweet.io`. The name “dweet” is based on “tweet”—a dweet is like a tweet from a device. Many of today's Internet-connected security systems include temperature sensors that can issue low-temperature warnings before pipes freeze or high-temperature warnings to indicate there might be a fire. Our simulated sensor will send dweets containing a location and temperature, as well as low- and high-temperature notifications. These will be `True` only if the temperature reaches 3 degrees Celsius or 35 degrees Celsius, respectively. In the next section, we'll use `freeboard.io` to create a simple dashboard that shows the temperature changes as the messages arrive, as well as warning lights for low- and high-temperature warnings.

Installing Dweepy

To publish messages to `dweet.io` from Python, first install the Dweepy library:

```
pip install dweepy
```

The library is straightforward to use. You can view its documentation at:

```
https://github.com/paddycarey/dweepy
```

Invoking the `simulator.py` Script

The Python script `simulator.py` that simulates our thermostat is located in the `ch16` example folder's `iot` subfolder. You invoke the simulator with two command-line arguments representing the number of total messages to simulate and the delay in seconds between sending dweets:

```
ipython simulator.py 1000 1
```

Sending Dweets

The `simulator.py` is shown below. It uses random-number generation and Python techniques that you've studied throughout this book, so we'll focus just on a few lines of code that publish messages to `dweet.io` via Dweepy. We've broken apart the script below for discussion purposes.